14 haftalık yol haritasında 1. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

# HAFTA 01

# Algoritmalara Dönüş: Doğruluk, Döngü Değişmezleri ve Deneysel Ölçüm

Hesaplama problemi ve sözleşme, Hoare üçlüsü, döngü değişmezi (başlatma–koruma–sonlanma), sonlanma kanıtı, assert ve kâhinle test, tekrarlanabilir ölçüm; C · C++ · Python

MODÜL I · ALGORİTMALARIN DİLİ · H1–H3

HAFTANIN BÜYÜK SORUSU

*Bir programın bütün testleri geçmesi, o programın doğru olduğunu kanıtlar mı?*

**Dr. Öğr. Üyesi Mehmet Sevri**
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

# İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları ... 3
2. AP-1'den AP-2'ye: Çalışan Programdan Doğru ve Verimli Algoritmaya ... 4
3. Kavramsal Anlatım ... 5
3.1 Hesaplama problemi, örnek ve algoritma ... 5
3.2 Doğruluğun iki bileşeni: kısmi doğruluk ve sonlanma ... 5
3.3 Sözleşme ile tasarım: ön koşul ve son koşul ... 5
3.4 Hoare üçlüsüne sezgisel bakış ... 5
3.5 Döngü değişmezi: başlatma, koruma, sonlanma ... 6
3.6 Değişmez bulma stratejileri ... 8
3.7 Sonlanma kanıtı: kesin azalan, alttan sınırlı nicelik ... 8
3.8 Test yaklaşımları: birimden strese ... 8
3.9 Deneysel ölçüm: saat, tekrar, ısınma, ortanca ... 9
3.10 Üç dilli çalışma düzeni ... 11
3.11 Tekrarlanabilir rastgele veri: rand() yerine ortak LCG ... 12
4. Haftanın Algoritmaları ... 12
4.1 En büyük eleman (Örnek 1 ve 2) ... 12
4.2 Doğrusal arama (Örnek 4) ... 16
4.3 Öklid algoritması: EBOB (Örnek 5) ... 19
5. Çözümlü Örnekler: Merdivenin Kalan Basamakları ... 22
6. Haftanın Deneyi: Aynı Algoritma, Üç Dil, İki Derleme Düzeyi ... 27
7. Hata Avcısı ... 30
8. Dil Penceresi: C · C++ · Python ... 32
9. Mühendis Gözüyle ... 33
10. Mini-OBS 2 Adımı: v2.0 — Veri Üreteci, Ölçüm Düzeni ve Kâhinle Test ... 34
11. Alıştırmalar ve Atölye ... 36
11.1 Alıştırmalar ... 36
11.2 Atölye ... 39
12. Kendini Değerlendirme ... 41
13. İleri Bakış ve Merak Kutusu ... 41
14. Özet, Kavram Haritası ve Terim Sözlüğü ... 42
15. Kaynaklar ... 44
Ek A. Dil Karşılıkları ... 45

# 1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

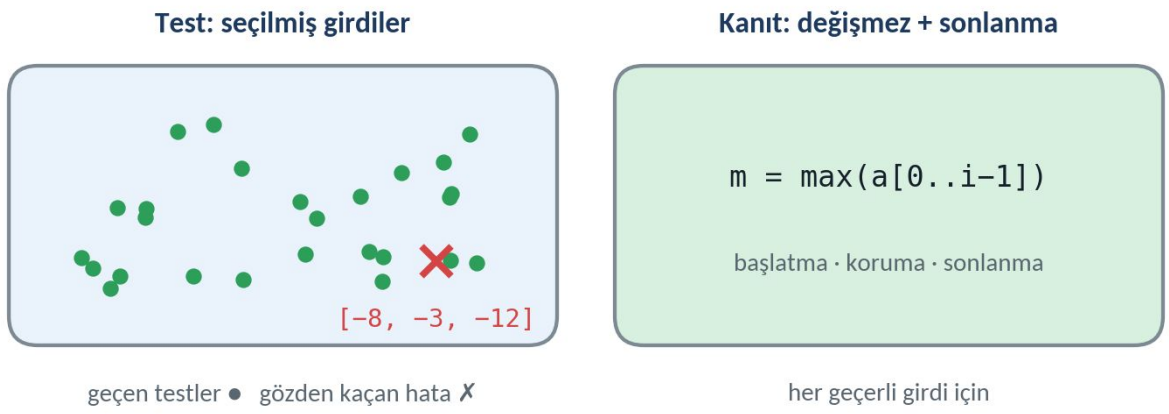
### Bir programın bütün testleri geçmesi, o programın doğru olduğunu kanıtlar mı?

AP-1'de bir programın "çalıştığını" birkaç girdiyle deneyerek gördük. AP-2'de soruyu keskinleştiriyoruz: program ön koşulu sağlayan **her** girdide doğru mu, bunu nasıl gerekçelendiririz; ne kadar hızlı çalıştığını ölçüm gürültüsüne aldanmadan nasıl ölçeriz? Bu hafta iki araç kuruyoruz: doğruluğun gerekçesi olan **döngü değişmezi** (loop invariant) ve verimliliğin ölçüsü olan **deney düzeni**. İkisinin arasında **testler** durur: hata bulmanın en ucuz yolu, ama kanıt değil.

> **GERÇEK DÜNYADAN · 2015: Timsort'ta yıllarca fark edilmeyen değişmez ihlali**
>
> Timsort, Python'un ve Java'nın (nesne dizileri için) standart sıralama algoritmasıdır: hazır sıralı parçaları (koşu, *run*) bulup birleştirir ve bekleyen koşuların uzunluklarını bir yığında (stack) tutar. Birleştirme kodu bu uzunlukların bir eşitsizlik zincirini, yani bir **değişmezi** koruduğunu varsayar. de Gouw ve arkadaşları OpenJDK gerçekleştirimini **KeY** biçimsel doğrulama aracıyla doğrulamaya çalışırken değişmezin her durumda korunmadığını buldular: kurgulanmış girdilerde Java'da dizi sınırı aşımı (`ArrayIndexOutOfBoundsException`) oluşabiliyordu. Aynı kusur Python gerçekleştiriminde de vardı ama pratikte erişilemeyecek büyüklükte girdi gerektiriyordu; ikisi de düzeltildi (de Gouw vd., 2015). Dijkstra'nın uyarısı öyküyü özetler: test, hataların varlığını gösterebilir; yokluklarını kanıtlayamaz (Dahl vd., 1972).

Büyük Soru'nun yanıtı **hayır**dır: testler yalnızca denenen girdiler hakkında bilgi verir (Şekil 1.1). Doğruluğun gerekçesi değişmez ve sonlanma kanıtıdır; testler bu gerekçenin koda doğru aktarılıp aktarılmadığını ve gözden kaçan varsayımları yakalar. İyi bir mühendis ikisini birlikte kullanır ve ölçümle destekler.

**Şekil 1.1**
*Test seçilmiş girdileri dener (biri hatayı kaçırır); değişmez ve sonlanma kanıtı her geçerli girdiyi kapsar*




### Öğrenme çıktıları

- **ÖÇ 1.1** Bir hesaplama problemini girdi kümesi, çıktı koşulu ve örnek (instance) kavramlarıyla tanımlar; algoritmayı ön koşul–son koşul sözleşmesiyle ifade eder. *(Bölüm 3.1–3.4, Örnek 1)*
- **ÖÇ 1.2** Döngü değişmezini başlatma–koruma–sonlanma adımlarıyla kurar ve basit algoritmaların (toplam, en büyük eleman, doğrusal arama, Öklid) doğruluğunu gerekçelendirir. *(Bölüm 3.5–3.7, 4.1–4.3, Örnek 2–5)*

- **ÖÇ 1.3** assert, sınır değer testleri ve kaba kuvvet bir kâhinle (test oracle) rastgele karşılaştırmalı testler yazar. *(Bölüm 3.8, Örnek 9–10, Bölüm 7)*
- **ÖÇ 1.4** Çalışma süresini C, C++ ve Python'da ölçer; tekrar, ısınma ve ortanca (medyan) kullanarak ölçüm gürültüsünü azaltan bir deney düzeni kurar. *(Bölüm 3.9, Örnek 11–13, Bölüm 6)*
- **ÖÇ 1.5** Aynı algoritmanın C, C++ ve Python karşılıklarını okur; üç dilde sözdizimi benzer ama anlamı farklı yapıları (tamsayı bölmesi, kalan, taşma) tanır. *(Bölüm 3.10, Örnek 6–7, Bölüm 8, Ek A)*

## 2. AP-1'den AP-2'ye: Çalışan Programdan Doğru ve Verimli Algoritmaya

AP-1'de C'nin yapı taşlarını kurduk ve dönemi Mini-OBS v1.0 ile bitirdik. AP-2 bu taşlarla **algoritma aileleri** kurar: arama ve sıralama, özyineleme ve geri izleme (backtracking), dinamik veri yapıları, karma tabloları, graflar, açgözlü algoritmalar ve dinamik programlama. Bu hafta sonraki her haftanın kullanacağı üç alışkanlığı kurar: **sözleşme ve değişmezle düşünmek, kâhinle test etmek, ölçerek karar vermek**. Ana dil C'dir; her örneğin C++ ve Python karşılığı da vardır (algoritma aynı, gösterim dile özgü) ve bunları okuyabilmeniz yeterlidir. Algoritmanın kendisi hiçbir dilde kütüphaneye devredilmez: `std::max_element` ya da `max` gibi çağrılar yalnızca kâhin (test oracle) olarak geçer.

**Tablo 1.1**
*AP-1'in yapı taşları ve bu haftadaki karşılıkları*

| AP-1'de | Bu hafta AP-2'de |
|---|---|
| Döngüler (H6–H7) | Döngü değişmezi: başlatma, koruma, sonlanma; azalan nicelikle sonlanma kanıtı |
| Fonksiyonlar (H8–H9) | Sözleşme: ön koşul çağıranın, son koşul fonksiyonun sorumluluğu; `assert` |
| Diziler (H10): `a[0..n−1]` | En büyük eleman ve doğrusal arama; "işlenmiş bölge" `a[0..i−1]` |
| İşaretçiler (H12): `const int a[]` | Diziyi değiştirmeyen sözleşme; çıktı parametresi `bol(x, y, &q, &r)` |
| Dosyalar (H14), Mini-OBS v1.0 | Ölçümlerin CSV'ye yazılması; Mini-OBS 2 v2.0: veri üreteci ve kâhin testi |

> **TAHMİN ET · Geri çağırma: üç soru (kodlar/quiz/; çalıştırmadan yanıtlayın)**
>
> - **S1 (AP-1 H10).** `H01_Q01_dizi_adim.c`: `int a[] = {3, 1, 4, 1, 5};` ve `int t = 0;` ile başlayan `for (int i = 0; i < 5; i += 2)` döngüsü her adımda `t += a[i];` yapıyor. Program ne yazdırır?
> - **S2 (AP-1 H8).** `H01_Q02_deger_aktarim.c`: `void arttir(int x)` içinde `x = x + 1;` yapılıyor; `main`'de `int x = 5;` iken `arttir(x);` çağrısından sonra `main`'deki `x` kaçtır?
> - **S3 (AP-1 H12).** `H01_Q03_isaretci.c`: `void ekle(int *p, int k)` gövdesinde `*p += k;` var; `int x = 3;` iken `ekle(&x, 4);` sonrası `x` kaçtır?

**Yanıtlar (gerçek çıktılar).** S1: `12` (a[0] + a[2] + a[4]). S2: "içeride x = 6", "main'de x = 5": argümanlar **değerle** aktarılır. S3: `7`: `ekle` adresi alır ve `*p += k` ile çağıranın değişkenine yazar. Bir fonksiyonun **neyi değiştirebileceği** de sözleşmenin parçasıdır: `const int a[]` "diziyi değiştirmem" sözüdür.

# 3. Kavramsal Anlatım

## 3.1 Hesaplama problemi, örnek ve algoritma

Bir **hesaplama problemi** (computational problem), girdi kümesini ve her girdi için kabul edilebilir çıktıları belirleyen bir girdi–çıktı ilişkisidir; tek bir somut girdiye **örnek** (instance) denir. **Algoritma**, her örnek için sonlu sayıda iyi tanımlı adımda istenen çıktıyı üreten yöntemdir; her örnekte duruyor ve doğru çıktıyı veriyorsa **doğrudur** (Cormen vd., 2022). Çıktı koşulunun iki parçalı olduğuna dikkat edin (Tablo 1.2): m dizide bulunmalı (yoksa 1000 de "her elemandan büyük-eşit" olurdu) ve her elemandan büyük-eşit olmalıdır. Bölüm 7'deki ilk hata birinci parçayı çiğner.

**Tablo 1.2**
*En büyük eleman problemi: girdi kümesi, işlem ve çıktı koşulu*

| GİRDİ | İŞLEM | ÇIKTI |
|---|---|---|
| • n ≥ 1 ve a[0..n−1] tamsayı dizisi<br>• örnek: a = [3, −1, 7, 2, 7, 5] | • diziyi bir kez soldan sağa tara<br>• şimdiye kadarki en büyüğü m'de tut | • m dizinin bir elemanıdır<br>• her k için a[k] ≤ m<br>• örnek için: m = 7 |

## 3.2 Doğruluğun iki bileşeni: kısmi doğruluk ve sonlanma

**Kısmi doğruluk** (partial correctness): ön koşulu sağlayan girdiyle başlayan algoritma **sonlanırsa** çıktısı son koşulu sağlar. **Sonlanma** (termination): her böyle girdide algoritma sonlu adımda durur. İkisi birlikte **tam doğruluktur** (total correctness). İkisi bağımsızdır: hiç durmayan bir döngü kısmi olarak doğrudur ama işe yaramaz; m = 0 ile başlayan en büyük fonksiyonu her zaman durur ama tamamı negatif dizide yanlıştır. Bu yüzden iki ayrı gerekçe kurarız: kısmi doğruluk için **döngü değişmezi**, sonlanma için **azalan nicelik**.

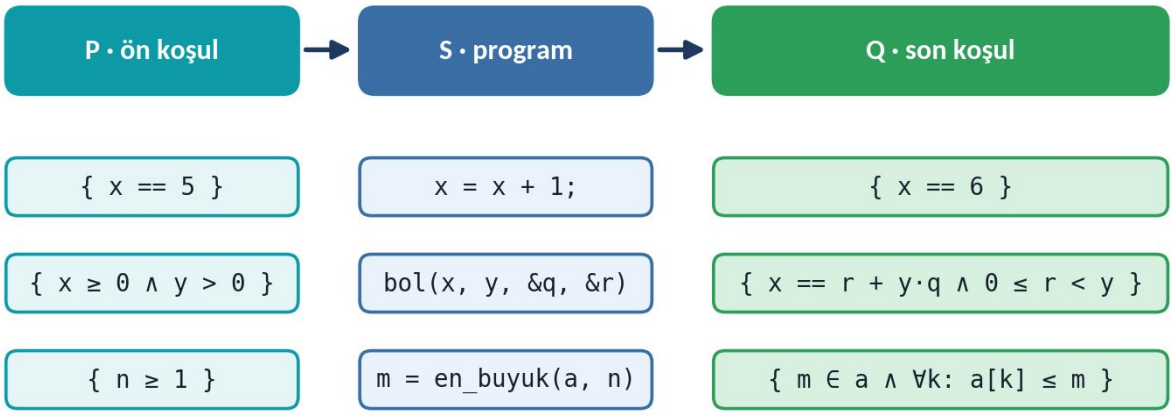
## 3.3 Sözleşme ile tasarım: ön koşul ve son koşul

**Sözleşme** (contract) iki koşuldur: **ön koşul** (precondition) çağrıdan önce doğru olmalıdır ve **çağıranın** sorumluluğudur; **son koşul** (postcondition) dönüşte doğru olacağı söz verilen koşuldur ve **fonksiyonun** sorumluluğudur. Sözleşmeyi yorumla yazar, geliştirme sırasında `assert` ile denetleriz. Örneğin `kavram/H01_K02_sozlesme_mutlak.c`'deki `mutlak(x)`'in ön koşulu `x != INT_MIN`'dir: −INT_MIN `int`'e sığmaz ve işaretli taşma **tanımsız davranıştır** (undefined behavior; HA4). Ön koşul ihlalini fonksiyonun içinde sessizce "düzeltmeyiz" (ör. 0 döndürmek): bu, çağıranın hatasını gizler. `assert` hatayı **yerinde** durdurur; kullanıcı girdisini doğrulamak ise `if` ile yapılan ayrı bir iştir.

## 3.4 Hoare üçlüsüne sezgisel bakış

Hoare (1969), bir programı **önünde ve arkasında doğru olan iddialarla** anlatan bir gösterim önerdi. Özgün makalede `P {Q} R` biçiminde yazılan **Hoare üçlüsü** (Hoare triple) bugün çoğunlukla **{P} S {Q}** yazılır: P doğruyken başlayan S **sonlanırsa** Q doğrudur; örneğin `{x == 5} x = x + 1; {x == 6}` (Şekil 1.2). Üçlü yalnızca kısmi doğruluğu söyler; sonlanma ayrıca gösterilir.

**Şekil 1.2**
*Hoare üçlüsü {P} S {Q}: ön koşul, program, son koşul (üç örneğin üçü de bu haftanın kodlarında assert ile denetlenir)*




Kısmi doğruluk: P'yi sağlayan bir durumda başlayan S SONLANIRSA Q sağlanır.

Tam doğruluk = kısmi doğruluk + sonlanma · Hoare üçlüsü (Hoare, 1969)

Hoare aksiyomlarını tamsayı bölmesini ardışık çıkarmayla yapan bir program üzerinde gösterir; `H01_K01_hoare_bolme.c` onun C karşılığıdır:

C **kavram/H01_K01_hoare_bolme.c · satır 13–24**

```c
void bol(int x, int y, int *q, int *r)
{
    assert(x >= 0 && y > 0);            /* P: ön koşul            */
    *r = x;
    *q = 0;
    while (y <= *r) {
        assert(x == *r + y * *q);       /* değişmez (döngü başı)  */
        *r = *r - y;
        *q = *q + 1;
    }
    assert(x == *r + y * *q && 0 <= *r && *r < y);   /* Q: son koşul */
}
```

ÇIKTI

```
$ ./H01_K01_hoare_bolme
17 = 5 * 3 + 2
 4 = 9 * 0 + 4
```

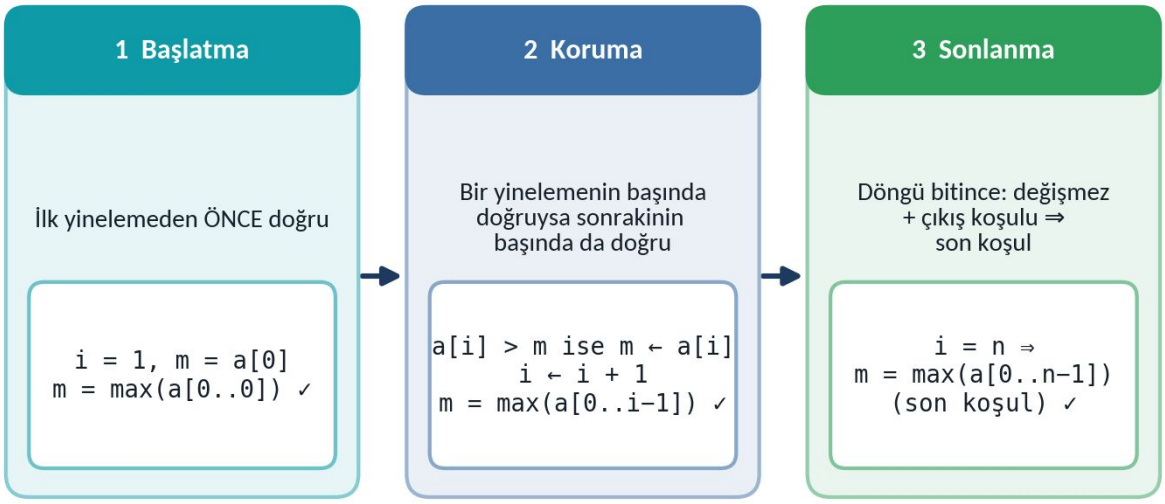
`x == r + y·q` döngü her başladığında doğrudur: bir **döngü değişmezidir**. Döngü bitince çıkış koşulu `y > r` de doğrudur; ikisi birlikte son koşulu verir. `y > 0` kısmi doğruluk için değil **sonlanma** için gerekir: y = 0 olsaydı r hiç azalmazdı.

## 3.5 Döngü değişmezi: başlatma, koruma, sonlanma

**Döngü değişmezi**, her yinelemenin **başında** doğru olan bir iddiadır ve üç adımla gösterilir (Cormen vd., 2022): **başlatma** (initialization): ilk yinelemeden önce doğrudur; **koruma** (maintenance): bir

yinelemenin başında doğruysa sonrakinin başında da doğrudur; **sonlanma** (termination): döngü bitince değişmez ile çıkış koşulu birlikte son koşulu verir (Şekil 1.3). İlk iki adım tümevarıma benzer; üçüncüsü onu döngüye bağlar. En basit örnek 1..n toplamıdır (Örnek 3); değişmez yorumda yazılıdır ve `izle(5)` iki tarafını her yinelemenin başında yazdırır:

**Şekil 1.3**
*Değişmez kanıtının üç adımı (örnek: en büyük eleman, m = max(a[0..i−1]))*




Tümevarıma benzer: başlatma = taban adımı, koruma = tümevarım adımı; sonlanma döngüye özgüdür

**C H01_03_toplam_1_n.c · satır 12–24**

```c
/* toplam_dongu: 1 + 2 + ... + n (ön koşul: n >= 0).
 * Değişmez (döngü başında): s = 1 + 2 + ... + (k-1) = (k-1)k/2
 * Başlatma : k = 1, s = 0 = 0·1/2.
 * Koruma   : s += k sonrası s = k(k+1)/2; k++ ile yine (k-1)k/2.
 * Sonlanma : döngü k = n+1'de biter → s = n(n+1)/2.            */
long long toplam_dongu(int n)
{
    long long s = 0;
    for (int k = 1; k <= n; k++) {
        s += k;
    }
    return s;
}
```

**ÇIKTI**

```
$ ./H01_03_toplam_1_n          # çıktının ilk bölümü
n = 5 için değişmez s = (k-1)k/2 (döngü başında):
  k = 1: s =  0, (k-1)k/2 =  0
  k = 2: s =  1, (k-1)k/2 =  1
  k = 3: s =  3, (k-1)k/2 =  3
  k = 4: s =  6, (k-1)k/2 =  6
  k = 5: s = 10, (k-1)k/2 = 10
  k = 6: s = 15, (k-1)k/2 = 15  <- k = n+1, döngü biter
```

## 3.6 Değişmez bulma stratejileri

Değişmez çoğunlukla **son koşuldan** türetilir. Üç strateji bu haftanın bütün döngülerini kapsar (Tablo 1.3): (1) **"şimdiye kadar"**: son koşul bütün dizi hakkındaysa değişmez işlenmiş bölge hakkında aynı şeyi söyler (`m = max(a[0..i−1])`; i = n'de son koşula dönüşür); (2) **son koşulun bir parçasını bırakmak**: bölmede `x = r + y·q` değişmez, `r < y` çıkış koşuludur; (3) **korunan nicelik**: değişkenler değişir ama bir fonksiyonları değişmez (Öklid'de ebob(a, b)).

**Tablo 1.3**
*Bu haftanın döngüleri: son koşul, değişmez ve strateji*

| Döngü | Son koşul | Değişmez (döngü başında) | Strateji |
|---|---|---|---|
| toplam 1..n | `s = n(n+1)/2` | `s = (k−1)k/2` | şimdiye kadar |
| en büyük | `m = max(a[0..n−1])` | `m = max(a[0..i−1])` | şimdiye kadar |
| doğrusal arama | `x ∉ a` ya da `a[i] = x (ilk)` | `x ∉ a[0..i−1]` | şimdiye kadar |
| bölme (Hoare) | `x = r + y·q ∧ 0 ≤ r < y` | `x = r + y·q ∧ r ≥ 0` | parçayı bırak |
| Öklid | `sonuç = ebob(a₀, b₀)` | `ebob(a, b) = ebob(a₀, b₀)` | korunan nicelik |

Sık yanlış: gövdenin **sonunda** doğru olan ifadeyi değişmez sanmak. `s = k(k+1)/2`, `s += k`'dan sonra doğrudur ama döngü başında (k = 1, s = 0) yanlıştır (HA2).

## 3.7 Sonlanma kanıtı: kesin azalan, alttan sınırlı nicelik

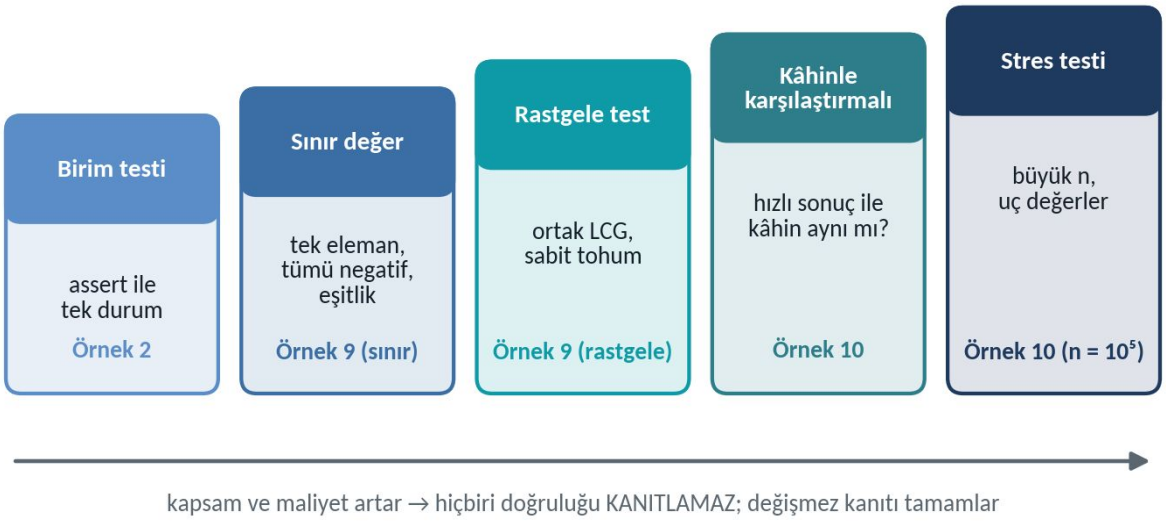
Sonlanmayı göstermenin standart yolu, her yinelemede **kesin azalan** ve **alttan sınırlı** bir tamsayı nicelik (sonlanma ölçütü, *variant*) bulmaktır: negatif olmayan bir tamsayı sonsuza dek azalamaz. `for (i = 0; i < n; i++)`'de nicelik `n − i`'dir; Öklid'de b'dir (yerine 0 ≤ a mod b < b gelir); bölmede r'dir ama yalnızca y > 0 iken azalır. Her döngü için böyle bir nicelik bulunamaz: Collatz döngüsünün her sayıda durduğu kanıtlanmış değildir (Bölüm 13).

## 3.8 Test yaklaşımları: birimden strese

Değişmez algoritmanın **fikrinin**, test ise **kodun** doğruluğunu sınar. Bu hafta beş basamaklı bir yelpaze kuruyoruz; kapsam ve maliyet basamaktan basamağa artar (Şekil 1.4):

- **Birim testi** (unit test; `assert`): tek bir durum; değişmez de assert'e dönüştürülebilir (Örnek 2).
- **Sınır değer testi** (boundary value testing): tek eleman, boş dizi, tamamı negatif, eşitlik, b = 0: hatalar sınırlarda yaşar (Örnek 9).
- **Rastgele test** (random testing): ortak LCG ve sabit tohumla çok sayıda girdi; girdi dağılımı hatayı kapsamazsa o da kördür (Örnek 9).
- **Kâhinle karşılaştırmalı test** (differential testing): hızlı fonksiyon, yavaş ama **apaçık doğru** ve **farklı yoldan** hesaplayan bir **kâhinle** (test oracle; çoğunlukla kaba kuvvet, brute force) aynı girdilerde karşılaştırılır; uyuşmazlıkta tohum ve **karşı örnek** (counterexample) raporlanır (Örnek 10).
- **Stres testi** (stress test): büyük n ve uç değerler (Örnek 10'da n = $10^5$). Testin kendisi de **kasıtlı hatalı** bir sürümle sınanır: bozuk kodu yakalamayan test gerçek hatayı da yakalamaz.

**Şekil 1.4**
*Test yelpazesi: birim → sınır → rastgele → kâhinle karşılaştırmalı → stres; her basamak bu haftanın bir örneğiyle, hiçbiri doğruluğu kanıtlamaz*




## 3.9 Deneysel ölçüm: saat, tekrar, ısınma, ortanca

Üç saat kaynağı vardır: **duvar saati** (wall-clock time) gerçek geçen süreyi verir ama sistem saati ayarlanınca geri ya da ileri atlayabilir; **monotonik saat** (monotonic clock) hiç geri gitmez (C++ `std::chrono::steady_clock`, Python `time.perf_counter`) ve süre ölçümü için doğru seçimdir; **işlemci zamanı** (CPU time, `clock()`) yalnızca sürecin işlemcide geçirdiği süreyi sayar. Tek ölçüme güvenilmez: ilk çalıştırma önbellekleri doldurur, işletim sistemi süreci bekletebilir. Ölçüm düzenimiz (Şekil 1.5): sabit tohumla veri; **1 ısınma** (warm-up; ölçülmez); **K tekrar** ve **ortanca** (median; aykırı gecikmelerden ortalamadan çok daha az etkilenir), en az ve en çokla birlikte; yayılım büyükse oturumu (ayrı süreç) tekrarla. Ölçülen işin **sonucu kullanılır** ve programlar **-O2** ile derlenir. Kuramsal kestirimi ölçümle sınamanın temeli bu düzendir (Sedgewick & Wayne, 2011).

**Şekil 1.5**
*Ölçüm düzeni: veri → ısınma → K tekrar → ortanca → CSV → grafik; yayılım büyükse oturum tekrarlanır*

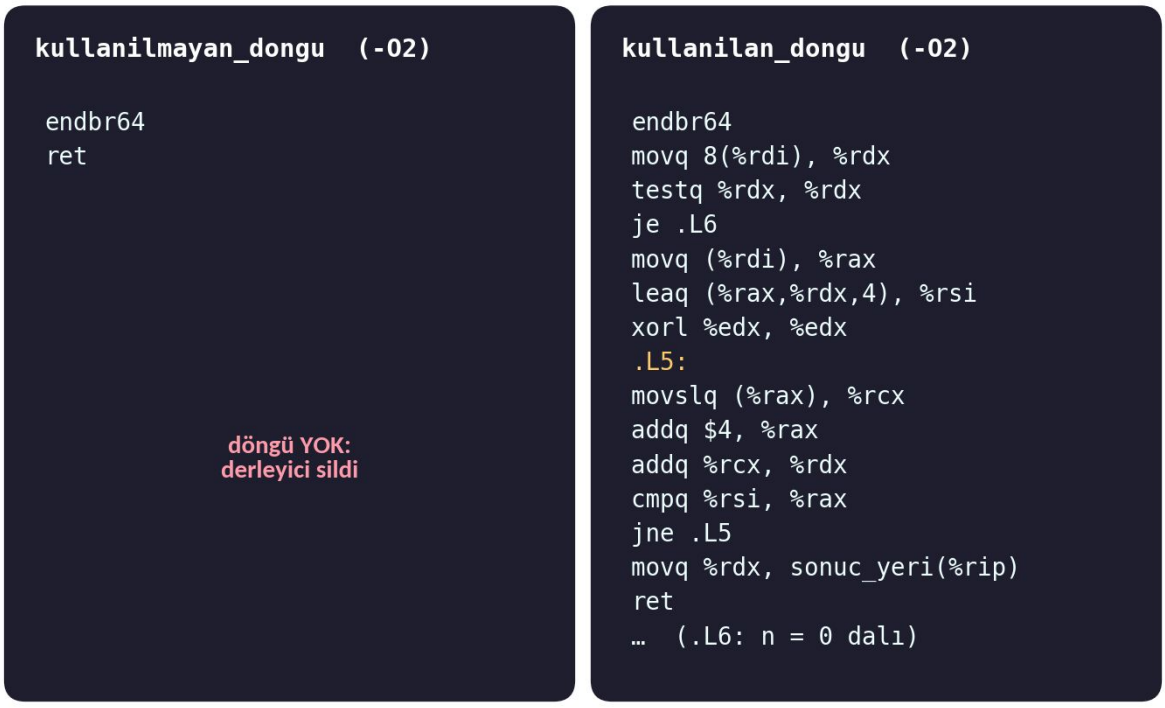



süreler stderr'e / CSV'ye · stdout'a yalnızca deterministik sonuçlar · -O2 (karşılaştırma için -O0)

**KAPUTUN ALTINDA · Derleyici deneyi nasıl bozar: sonucu kullanılmayan döngü**

"Sanki" (as-if) kuralına göre derleyici, **gözlemlenebilir davranışı** değiştirmediği sürece kodu dönüştürebilir. Toplamı hiçbir yere yazmayan döngünün gözlemlenebilir etkisi yoktur; gcc 13 -O2 onu tümüyle siler (Şekil 1.6). Sonuç dış bir değişkene yazılınca döngü kalır. Ölçülen işin sonucunu kullanın ve şüpheli kısa süreleri denetleyin (Örnek 12); aynı denemeyi Compiler Explorer'da (godbolt.org) da yapabilirsiniz.

**Şekil 1.6**
*gcc -O2 çıktısı (Örnek 12): solda sonucu kullanılmayan döngü (yalnızca ret), sağda sonucu yazılan döngü*




gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0 -O2 · sonuç yazılınca (sonuc_yeri) döngü kalır: 5 komutluk iç döngü

**KAPUTUN ALTINDA · Ölçülen süre nereden gelir?**

C11'in `timespec_get(&t, TIME_UTC)` fonksiyonu duvar saatini saniye ve nanosaniye olarak verir; monotonik değildir: ölçüm sırasında sistem saati (ör. NTP ile) ayarlanırsa tek bir ölçüm bozulabilir. Standart C'de monotonik saat yoktur (POSIX'te `clock_gettime(CLOCK_MONOTONIC, …)` vardır); C++ ve Python sürümlerimiz monotonik saat kullanır. Ölçüm gürültüsünün asıl kaynağı ise işletim sisteminin süreci zaman zaman bekletmesi ve arka plandaki süreçlerin çekirdeği ve önbelleği paylaşmasıdır.

Saatin **çözünürlüğü** de önemlidir: süreyi saniye cinsinden bir `double`'da tutmak, değer büyüdükçe ayırt edilebilen en küçük adımı büyütür. `kavram/H01_K04_double_cozunurluk.c` bu adımı `nextafter(x, INFINITY) - x` ile hesaplar: 1 s'de $2{,}2\cdot10^{-16}$ s, 1000 s'de $1{,}1\cdot10^{-13}$ s, ama 1970'ten beri geçen ≈ $1{,}76\cdot10^{9}$ s'de 238,4 ns. Bu yüzden ortak `zamanlayici` modülünün `zaman_sn()`'i ilk çağrıdaki saniyeyi taban alır ve ilk çağrıya göreli süreyi döndürür. Modülün ilk taslağında bu taban yoktu ve deneme ölçümlerinde Örnek 12'nin süresi "0 ms" çıkıyordu: ölçüm aracının kendisi de sınanmalıdır. Bu belgedeki ölçümler düzeltilmiş modülle yapıldı.

## 3.10 Üç dilli çalışma düzeni

Her örnek üç dosyadır: `kodlar/` altındaki `c/`, `cpp/` ve `python/` klasörlerinde aynı gövde adıyla (`HNN_SS_ad.c`, `.cpp`, `.py`; SS örnek numarası). Fonksiyon ve değişken adları üç dilde aynıdır (Türkçe, ASCII, `snake_case`; Python dosyaları PEP 8'e uyar: van Rossum vd., 2001); böylece yan yana okurken yalnızca **gerçek dil farkları** göze çarpar. Web sayfasındaki kod kutuları bu üç dosyayı C | C++ | Python sekmelerinde gösterir ve "farkları göster" düğmesi C'den farklılaşan satırları hardal renkle vurgular; Ek A'daki kod kutuları aynı vurguyu kullanır. Komutlar Tablo 1.4'tedir ve her dosyanın başlık yorumunda yazılıdır.

**Tablo 1.4**
*Derleme ve çalıştırma komutları (gcc/g++ 13, Python 3.11 ile sınandı)*

| Dil | Derleme / çalıştırma | Not |
|---|---|---|
| C (C17) | `gcc -std=c17 -Wall -Wextra -pedantic -O2 ad.c -o ad` | ortak modül kullananlarda ortak_rastgele.c / zamanlayici.c eklenir |
| C++ (C++17) | `g++ -std=c++17 -Wall -Wextra -pedantic -O2 ad.cpp -o ad` | ortak modüller yalnız başlık (.hpp) |
| Python 3 | `python3 ad.py`  (Windows: `py ad.py`) | 3.10 ve üstü; yalnızca standart kütüphane |

Eşdeğerliği `kodlar/` klasöründeki `python3 esdeger_test.py .` denetler: her örneği üç dilde derler, çalıştırır ve çıktıları `ciktilar/` altındaki beklenen çıktılarla bayt bayt karşılaştırır. "=" örneklerinde üç çıktı aynıdır; "Δ" örneklerinde (6 ve 7) diller bilinçli olarak farklıdır ve her dilin beklenen çıktısı ayrı dosyadadır. Bu haftanın sonucu: "Toplam 13 örnek · geçen 13 · hatalı 0".

> KAPUTUN ALTINDA · **Aynı algoritma, üç maliyet modeli**
>
> C ve C++ makine koduna derlenir: toplam döngüsünün gövdesi gcc 13 -O2 ile **5 makine komutudur**. CPython kaynağı **bayt koduna** (bytecode) çevirir ve her komutu yorumlayıcı döngüsünde çözer; her eleman bir Python nesnesidir. `dis` modülü bayt kodunu gösterir (aşağıda Python 3.11.15 çıktısının döngü bölümü). n = $10^6$'da eleman başına süre C -O2'de 0,47 ns, Python'da 38,4 ns ölçüldü (Bölüm 6): ≈ 81 kat. Asimptotik sınıf aynıdır (Θ(n)); sabit çarpanlar bir ila iki mertebe farklıdır. Bayt kodu sürüme göre değişir: 3.10'da `BINARY_OP` yerine `INPLACE_ADD`, `JUMP_BACKWARD` yerine `JUMP_ABSOLUTE` görülür.

ÇIKTI · PYTHON (DİS)

```
$ python3 H01_K05_dis_toplam.py      # döngü bölümü
        >>   62 FOR_ITER                13 (to 90)
             64 STORE_FAST               2 (i)

 15          66 LOAD_FAST                1 (s)
             68 LOAD_FAST                0 (a)
             70 LOAD_FAST                2 (i)
             72 BINARY_SUBSCR
             82 BINARY_OP               13 (+=)
             86 STORE_FAST               1 (s)
             88 JUMP_BACKWARD           14 (to 62)
```

### 3.11 Tekrarlanabilir rastgele veri: rand() yerine ortak LCG

Bir deney ancak aynı girdiyle tekrarlanabilir. C'nin `rand()`'ı buna uygun değildir: standart algoritmayı belirlemez (yalnızca `RAND_MAX` ≥ 32767 ister) ve aynı tohum farklı kitaplıklarda farklı dizi verir (HA6); `std::mt19937` ile Python'un `random`'u da aynı tohumla birbirinin dizisini vermez. Ders boyunca sabit bir **tohumla** (seed) başlatılan **ortak doğrusal eşlik üreteci** (linear congruential generator, LCG) kullanılır: 64 bitlik durum her adımda `durum · A + C (mod 2⁶⁴)` olur, çıktı durumun üst 31 bitidir (A = 6364136223846793005, C = 1442695040888963407). Örnek 8 formülü önce elle uygular; aynı tohum üç dilde bayt bayt aynı diziyi verir:

C `H01_08_ortak_lcg.c · satır 29–34`

```c
    /* 1) Formülü elle uygula: durum = durum * A + C (mod 2^64) */
    uint64_t durum = 42;
    durum = durum * UINT64_C(6364136223846793005)
            + UINT64_C(1442695040888963407);   /* uint64_t: 2^64 modunda */
    printf("Elle   : durum = %" PRIu64 "\n", durum);
    printf("         çıktı = durum >> 33 = %" PRIu64 "\n", durum >> 33);
```

ÇIKTI

```
$ ./H01_08_ortak_lcg              # C, C++ ve Python'da aynı çıktı
Elle   : durum = 10481999410520546993
         çıktı = durum >> 33 = 1220265334
Modül  : rastgele_sayi() x 4 = 1220265334 484179026 886563538 1353769503
rastgele_aralik(1, 6) x 10  = [3, 5, 6, 1, 3, 6, 5, 3, 1, 5]
rastgele_kesir() x 3        = 0.795883 0.390873 0.788332
rastgele_dizi(8, -50, 50)   = [46, 42, 18, 5, -14, 12, 39, -33]
rastgele_karistir(1..10)    = [6, 1, 2, 3, 5, 9, 4, 7, 10, 8]
Yeniden tohum 42, ilk sayı  = 1220265334 (aynı dizi baştan)
```

C'de `uint64_t` çarpımı $2^{64}$ modunda kendiliğinden sarar; Python'da aynı adım `& (2**64 - 1)` maskesiyle yapılır. Tohum deney kaydının parçasıdır: çıktıya ve CSV'ye yazılır.

## 4. Haftanın Algoritmaları

Üç amiral algoritmayı aynı on bir adımla işliyoruz: problem → fikir → elle izleme → sözde kod → doğruluk → analiz ve kimlik kartı → C gerçekleştirimi → üç dil karşılaştırması → test → deney → varyantlar. Bu sıra dönem boyunca her algoritmada yinelenecek.

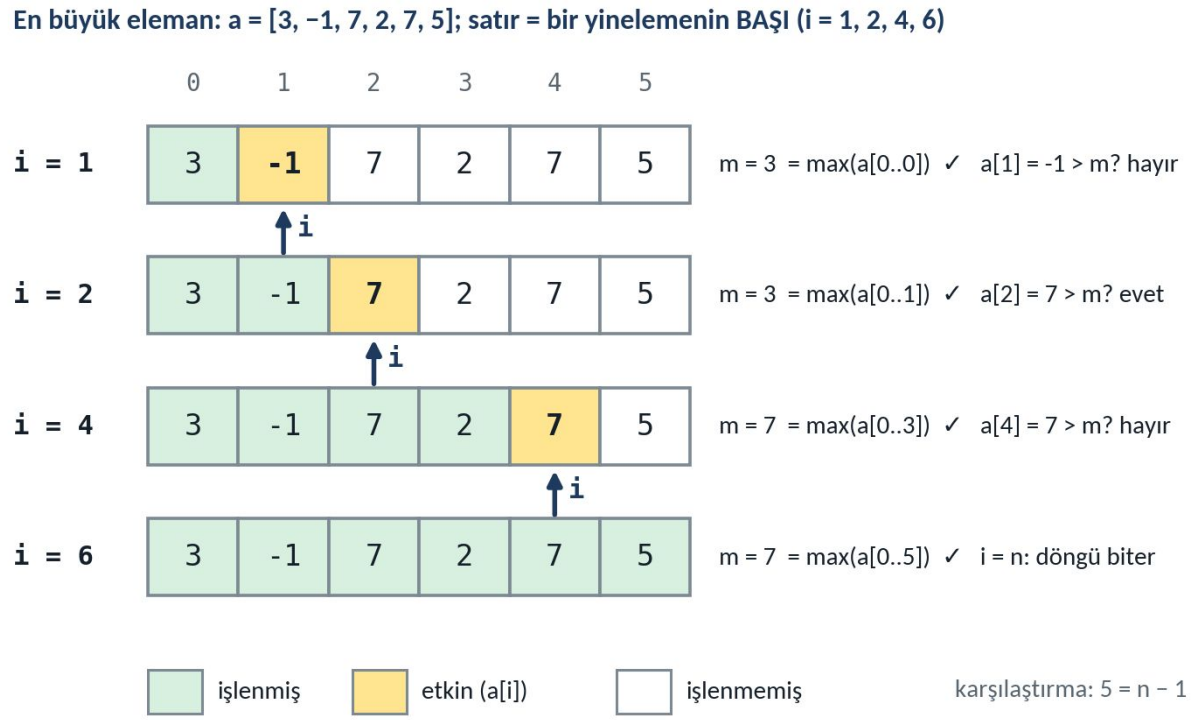
### 4.1 En büyük eleman (Örnek 1 ve 2)

**1. Problem.** n ≥ 1 elemanlı a[0..n−1] için dizinin bir elemanı olan ve her elemandan büyük-eşit m (Tablo 1.2). Örnek: a = [3, −1, 7, 2, 7, 5] için m = 7.

**2. Fikir ve analoji.** Sınıfın en uzun boylu öğrencisini bulmak için öğrencileri sırayla geçer, o ana kadarki en uzunu aklımızda tutarız. Rekoru **gerçek bir elemanla** (a[0]) başlatırız: hiçbir şey görmemişken 0 gibi uydurma bir değer rekor olamaz.

**3. Elle izleme.** Şekil 1.7'nin her satırı bir yinelemenin **başıdır** (dört seçilmiş yineleme): açık yeşil işlenmiş bölge a[0..i−1], sarı etkin hücre a[i]. Tablo 1.5 aynı izlemenin programla üretilmiş sayılarıdır (karşılaştırma sayısı 5 = n − 1).

**Şekil 1.7**
*Değişmez şeridi: en büyük eleman, a = [3, −1, 7, 2, 7, 5], seçilmiş dört yineleme; her satırda m = max(işlenmiş bölge)*




**Tablo 1.5**
*En büyük eleman izleme tablosu (a = [3, −1, 7, 2, 7, 5])*

| i (başta) | işlenmiş bölüm | m (başta) | m = max? | a[i] | a[i] > m? | m (sonra) |
|---|---|---|---|---|---|---|
| 1 | `a[0..0] = [3]` | 3 | evet | -1 | hayır | 3 |
| 2 | `a[0..1] = [3, -1]` | 3 | evet | 7 | evet | **7** |
| 3 | `a[0..2] = [3, -1, 7]` | 7 | evet | 2 | hayır | 7 |
| 4 | `a[0..3] = [3, -1, 7, 2]` | 7 | evet | 7 | hayır | 7 |
| 5 | `a[0..4] = [3, -1, 7, 2, 7]` | 7 | evet | 5 | hayır | 7 |
| 6 | `a[0..5] = [3, -1, 7, 2, 7, 5]` | 7 | evet | — | — | döngü biter |

**TAHMİN ET · Sizce?**

- i = 4'te a[4] = 7 = m. Karşılaştırma >= olsaydı sonuç değişir miydi? (Değer aynı kalır; konumu da döndüren sürümde >= son konumu verir: Alıştırma 1.1.) Tamamı negatif b = [−8, −3, −12] için m kaç olur; m = 0 ile başlasaydık? (−3; 0 ile başlayan sürüm 0 döndürür: HA1.)

**4. Sözde kod.**

SÖZDE KOD · En büyük eleman

```
algoritma EN-BÜYÜK(a[0..n−1])              ▷ ön koşul: n ≥ 1
    m ← a[0]
    i ← 1'den n−1'e kadar her i için        ▷ değişmez: m = max(a[0..i−1])
        eğer a[i] > m ise
            m ← a[i]
    döndür m                                ▷ son koşul: m = max(a[0..n−1])
```

**5. Doğruluk. Değişmez:** her yinelemenin başında m = max(a[0..i−1]) ve m bu bölümün bir elemanıdır. **Başlatma:** i = 1, m = a[0]: a[0..0]'ın en büyüğü a[0]'dır. **Koruma:** a[i] > m ise m ← a[i] olur ve a[i] bölümün tüm elemanlarından büyüktür; değilse m, a[i]'den de büyük-eşittir. İki durumda da gövde sonunda m = max(a[0..i]) olur ve i ← i + 1 değişmezi yeniden kurar. **Sonlanma:** döngü i = n'de biter; değişmez m = max(a[0..n−1]) der. **Sonlanma kanıtı:** n − i her yinelemede 1 azalır ve 0'ın altına inmez.

**6. Analiz ve kimlik kartı.** Her girdide tam n − 1 karşılaştırma: en iyi, ortalama ve en kötü durum (best, average, worst case) **Θ(n)**; ek bellek **O(1)**. Daha iyisi yoktur: bakılmayan bir eleman en büyük olabilirdi.

**Tablo 1.6**
*Algoritma kimlik kartı: en büyük eleman*

| ALGORİTMA KİMLİK KARTI **En büyük eleman** *(maximum element, linear scan)* | |
|---|---|
| **Strateji** | Kaba kuvvet (tek geçiş) |
| **Girdi önkoşulu** | Dizi boş değil (n ≥ 1); sıralı olması gerekmez |
| **Zaman (en iyi · ortalama · en kötü)** | Θ(n) · Θ(n) · Θ(n) |
| **Ek bellek** | O(1) |
| **Özellikler** | Her girdide tam n − 1 karşılaştırma; diziyi değiştirmez; başlangıç değeri a[0] olmalı (0 değil: tamamı negatif dizide hata) |
| **Değişmez / doğruluk** | Döngü başında m = max(a[0..i−1]); i = n'de m = max(a[0..n−1]) |
| **Kütüphane karşılığı** | C: yok (elle yazılır) · C++: `*std::max_element(a.begin(), a.end())` · Python: `max(a)` |
| **Ne zaman kullanılır** | Sırasız veride en büyük (ya da < ile en küçük) değer bir kez aranacaksa |
| **Ne zaman kullanılmaz** | En büyük sürekli değişen veride sık sık sorulacaksa (öbek / öncelik kuyruğu, H7); dizi zaten sıralıysa son eleman O(1) |

**7. C gerçekleştirimi.** Sözleşme yorumdadır; n = 0 çağıranın hatasıdır (`a[0]`'ı okumak tanımsız davranış). `const int a[]` diziyi değiştirmeme sözüdür.

C **H01_01_en_buyuk.c · satır 12–28** gcc -std=c17 -Wall -Wextra -pedantic -O2 H01_01_en_buyuk.c -o H01_01_en_buyuk

```c
/* en_buyuk: a[0..n-1] dizisinin en büyük elemanını döndürür.
 * Ön koşul : n >= 1 (dizi boş değil); a en az n elemanlıdır.
 * Son koşul: dönen m dizinin bir elemanıdır ve her k için
 *            (0 <= k < n) a[k] <= m sağlanır.
 * Ön koşul ihlali (n == 0) çağıranın hatasıdır: a[0] geçerli bir
 * eleman değildir, okunması tanımsız davranıştır; fonksiyon bunu
 * ayrıca denetlemez.                                              */
int en_buyuk(const int a[], size_t n)
{
    int m = a[0];
    for (size_t i = 1; i < n; i++) {
        if (a[i] > m) {
            m = a[i];
        }
    }
    return m;
}
```

ÇIKTI · ÜÇ DİLDE AYNI

```
$ ./H01_01_en_buyuk
a = [3, -1, 7, 2, 7, 5] -> en büyük: 7
b = [-8, -3, -12] -> en büyük: -3
c = [42] -> en büyük: 42
```

**8. Üç dil karşılaştırması.** C++ diziyi `const std::vector<int>&` olarak alır (kopyalamadan, değiştirmeden); Python listesi boyutunu taşır ve `range(1, len(a))` aynı indisleri dolaşır. Hardal satırlar C'den farklılaşan satırlardır:

C++ **H01_01_en_buyuk.cpp · satır 21–31**

```cpp
int en_buyuk(const std::vector<int>& a)
{
    const std::size_t n = a.size();
    int m = a[0];
    for (std::size_t i = 1; i < n; i++) {
        if (a[i] > m) {
            m = a[i];
        }
    }
    return m;
}
```

Python **H01_01_en_buyuk.py · satır 16–22**

```python
def en_buyuk(a):
    """a'nın en büyük elemanı."""
    m = a[0]
    for i in range(1, len(a)):
        if a[i] > m:
            m = a[i]
    return m
```

**9. Test: sınır durumları ve değişmezin assert ile denetlenmesi.** Örnek 1'in girdileri bilinçli seçildi: karışık (en büyük iki kez), **tamamı negatif**, **tek elemanlı**. Örnek 2'de `degismez(a, i, m)` m'nin a[0..i−1]'in en büyüğü ve elemanı olduğunu Θ(i) sürede denetler; `assert` başlatmada, her `i++`'dan sonra ve sonlanmada çağrılır:

C **H01_02_en_buyuk_assert.c · satır 37–53**

```c
/* Ön koşul: n >= 1. Son koşul: m = max(a[0..n-1]). */
int en_buyuk(const int a[], size_t n)
{
    assert(n >= 1);                    /* ön koşul */
    int m = a[0];
    size_t i = 1;
    assert(degismez(a, i, m));         /* başlatma */
    while (i < n) {
        if (a[i] > m) {
            m = a[i];
        }
        i++;
        assert(degismez(a, i, m));     /* koruma */
    }
    assert(i == n);                    /* sonlanma: i == n */
    return m;  /* değişmez + (i == n)  =>  m = max(a[0..n-1]) */
}
```

Normal derlemede son satır "Değişmez assert ile 10 kez denetlendi (başarısız olsaydı program dururdu)" der; `-DNDEBUG` ile (`python3 -O` ile) assert'ler hiç değerlendirilmez ve aynı satır "Değişmez assert ile 0 kez denetlendi (başarısız olsaydı program dururdu)" olur. Sayaç bilerek assert'in içindedir. Ders: **assert içine programın asıl işi konmaz.**

**10. Deney.** `H01_A03_sayaclar.c` n = 10 için "en_buyuk, n = 10: 9 karşılaştırma" yazar (kuram: n − 1). Erişim deseni Haftanın Deneyi'ndeki dizi toplamıyla aynıdır (Bölüm 6).

**11. Varyantlar ve sonraki basamak.** En küçük eleman (> → <; Alıştırma 1.10); en büyüğün ilk (>) ya da son (>=) indisi; ikinci en büyük (iki rekor; Örnek 10); en küçük ve en büyük birlikte: elemanları ikişer işleyerek 2(n − 1) yerine en çok 3⌊n/2⌋ karşılaştırma (Cormen vd., 2022; Alıştırma 1.12).

## 4.2 Doğrusal arama (Örnek 4)

**1. Problem.** Girdi: a[0..n−1] (n = 0 olabilir; sıralı olmak zorunda değil) ve x. Çıktı: x varsa **ilk** konumu i (a[i] = x ve x ∉ a[0..i−1]); yoksa C/C++'ta −1, Python'da `None`. "Bulunamadı" değeri sözleşmenin parçasıdır: çağıran onu denetlemek zorundadır.

**2. Fikir ve analoji.** Sırasız bir rafta kitabı ararken kapakları soldan sağa okur, bulduğumuz **anda** dururuz; rafın sonuna gelirsek "yok" deriz. Her bakılan kitap, aranan olmadığını öğrendiğimiz kitaptır: **elenmiş bölge** büyür.

**3. Elle izleme.** a = [7, 3, 9, 3, 5, 1, 8] için x = 3 ikinci karşılaştırmada bulunur; x = 6 yoktur ve 7 karşılaştırmadan sonra i = n olur (Tablo 1.7; şerit gösterimi sunumda ve web oynatıcısında). **Sizce?** x = 8 ve x = 7 aranırsa kaç karşılaştırma yapılır; boş dizide döngü kaç kez döner? (Yanıtlar aşağıdaki çıktıda.)

**Tablo 1.7**
*Doğrusal arama izleme tablosu: x = 3 (bulundu) ve x = 6 (yok)*

| x | i | a[i] | a[i] == x? | karşılaştırma | değişmez (döngü başı) |
|---|---|---|---|---|---|
| 3 | 0 | 7 | hayır | 1 | a[0..-1] boş: 3 yok (doğru) |

| x | i | a[i] | a[i] == x? | karşılaştırma | değişmez (döngü başı) |
|---|---|---|---|---|---|
| 3 | 1 | 3 | **evet** | 2 | 3 ∉ a[0..0] |
| 6 | 0 | 7 | hayır | 1 | a[0..-1] boş: 6 yok (doğru) |
| 6 | 1 | 3 | hayır | 2 | 6 ∉ a[0..0] |
| 6 | 2 | 9 | hayır | 3 | 6 ∉ a[0..1] |
| 6 | 3 | 3 | hayır | 4 | 6 ∉ a[0..2] |
| 6 | 4 | 5 | hayır | 5 | 6 ∉ a[0..3] |
| 6 | 5 | 1 | hayır | 6 | 6 ∉ a[0..4] |
| 6 | 6 | 8 | hayır | 7 | 6 ∉ a[0..5] |

### 4. Sözde kod.

SÖZDE KOD · Doğrusal arama

```
algoritma DOĞRUSAL-ARA(a[0..n−1], x)
    i ← 0'dan n−1'e kadar her i için          ▷ değişmez: x ∉ a[0..i−1]
        eğer a[i] = x ise
            döndür i                          ▷ 1. çıkış: bulundu (ilk konum)
    döndür −1                                 ▷ 2. çıkış: x ∉ a[0..n−1]
```

**5. Doğruluk. Değişmez:** her yinelemenin başında x ∉ a[0..i−1]. **Başlatma:** i = 0'da bölüm boştur. **Koruma:** gövdede ya a[i] = x'tir ve fonksiyon döner, ya da x ∉ a[0..i] olur ve i ← i + 1 değişmezi korur. **İki çıkış, iki gerekçe:** 1. çıkışta a[i] = x ve x ∉ a[0..i−1]: i **ilk** konumdur; 2. çıkışta i = n'dir ve değişmez x ∉ a[0..n−1] der. **Sonlanma:** n − i her yinelemede azalır.

**6. Analiz ve kimlik kartı.** x ilk elemansa 1 karşılaştırma (**en iyi durum, O(1)**); x yoksa ya da sondaysa n (**en kötü durum, Θ(n)**). x dizide varsa ve her konum eşit olasılıklıysa ortalama (1 + 2 + … + n)/n = (n + 1)/2: **Θ(n)**. Örnek 4'ün sayacı: ilk eleman 1, yok 7 karşılaştırma.

**Tablo 1.8**
*Algoritma kimlik kartı: doğrusal arama*

| ALGORİTMA KİMLİK KARTI **Doğrusal arama** *(linear search)* | |
|---|---|
| **Strateji** | Kaba kuvvet (tek geçiş, erken çıkış) |
| **Girdi önkoşulu** | Yok — dizi sıralı olmak zorunda değil; boş dizi de geçerli |
| **Zaman (en iyi · ortalama · en kötü)** | O(1) · Θ(n) · Θ(n) |
| **Ek bellek** | O(1) |
| **Özellikler** | İki çıkış noktası: ilk konumu döndürür; yoksa −1 (C/C++) ya da None (Python); aranan yoksa n karşılaştırma |
| **Değişmez / doğruluk** | Döngü başında aranan a[0..i−1] içinde yok |
| **Kütüphane karşılığı** | C: standart C'de yok (POSIX `lfind`) · C++: `std::find` · Python: `a.index(x)` (yoksa `ValueError`), `x in a` |
| **Ne zaman kullanılır** | Küçük ya da sırasız veride, tek seferlik aramada; sıralama maliyetine değmeyecek durumlarda |

| Ne zaman kullanılmaz | Aynı veride çok sayıda arama yapılacaksa: önce sıralayıp ikili arama (H4) ya da karma tablosu (H10) |
|---|---|

**7. C gerçekleştirimi.** Dosyadaki yorum (satır 15–21) sözleşmeyi, değişmezi ve sonlanma gerekçesini yukarıdaki gibi yazar. `size_t` indis `int`'e dönüştürülerek döndürülür; −1 yalnızca işaretli türde anlamlıdır. `k == -1` denetlenmeden `a[k]` yazmak dizi dışına erişimdir (tanımsız davranış).

C **H01_04_dogrusal_arama.c · satır 22–32** gcc -std=c17 -Wall -Wextra -pedantic -O2 H01_04_dogrusal_arama.c -o H01_04_dogrusal_arama

```c
int dogrusal_ara(const int a[], size_t n,
                 int x)
{
    for (size_t i = 0; i < n; i++) {
        karsilastirma++;
        if (a[i] == x) {    /* 1. çıkış */
            return (int)i;
        }
    }
    return -1;              /* 2. çıkış */
}
```

ÇIKTI · ÜÇ DİLDE AYNI

```
$ ./H01_04_dogrusal_arama
a = [7, 3, 9, 3, 5, 1, 8] (n = 7)
ara(7) -> 0, karşılaştırma: 1
ara(3) -> 1, karşılaştırma: 2
ara(8) -> 6, karşılaştırma: 7
ara(6) -> bulunamadı, karşılaştırma: 7
boş dizi:
ara(7) -> bulunamadı, karşılaştırma: 0
```

**8. Üç dil karşılaştırması.** C++ sürümü C ile satır satır aynı yapıdadır; diziyi `const std::vector<int>&` olarak alır ve sınırı `a.size()`'dan okur (Ek A). Python sürümü "bulunamadı" için `None` döndürür ve çağıran `is None` ile sınar:

Python **H01_04_dogrusal_arama.py · satır 20–27**

```python
def dogrusal_ara(a, x):
    """x'in ilk konumu; yoksa None."""
    global karsilastirma
    for i in range(len(a)):
        karsilastirma += 1
        if a[i] == x:    # 1. çıkış
            return i
    return None          # 2. çıkış
```

**9. Test.** Boş dizi (0 karşılaştırma), aranan ilk ve son elemanda, **yinelenen** eleman (3 iki kez geçer: ilk konum 1 olmalı) ve olmayan eleman. Yinelenen elemanlı test önemlidir: "son konumu" döndüren hatalı sürüm elemanları farklı dizilerde bütün testleri geçer (Alıştırma 1.8). Kâhin: C++'ta `std::find`, Python'da `a.index(x)`, yalnızca testte.

**10. Deney.** Ortalama durumu sınamak için x'i dizinin elemanlarından rastgele seçin ve sayaç ortalamasını (n + 1)/2 ile karşılaştırın; Mini-OBS 2'nin `olcer arama` ölçümü aynı algoritmanın kayıt sayısıyla büyüyen süresini verir (Bölüm 10).

**11. Varyantlar ve sonraki basamak.** Son konum, sayma, koşulu sağlayan ilk eleman: değişmez aynı kalıptadır. Dizi **sıralıysa** her adımda aralığın yarısını eleyen **ikili arama** Θ(log n)'dir (H4); çok sayıda arama için sıralama (H5–H6) ya da karma tablosu (H10).

## 4.3 Öklid algoritması: EBOB (Örnek 5)

**1. Problem.** Girdi: a, b ≥ 0 tamsayıları, ikisi birden 0 değil. Çıktı: ikisini de bölen en büyük tamsayı ebob(a, b) (*greatest common divisor*). Örnek: ebob(252, 105) = 21.

**2. Fikir ve analoji.** a ve b'yi bölen her sayı a − q·b'yi de böler, tersi de doğrudur: ebob(a, b) = ebob(b, a mod b) ve problem her adımda **küçülür**. Geometrik analoji: 252 × 105'lik dikdörtgeni en büyük kare fayanslarla döşemek; 105 × 105'lik kareleri kesip atınca kalan 42 × 105'lik dikdörtgeni aynı fayans döşer.

**3. Elle izleme.** Tablo 1.9, Örnek 5'in izleme çıktısıyla aynıdır. **Sizce?** ebob(55, 89)'da a < b'dir; ilk adımda ne olur? (55 mod 89 = 55: a ile b yer değiştirir, ayrıca karşılaştırma gerekmez; Örnek 5'e göre ebob(55, 89) 10, ebob(89, 55) 9 adım sürer.)

**Tablo 1.9**
*Öklid izleme tablosu: ebob(252, 105) = 21 (3 adım) ve klasik örnek ebob(1071, 462) = 21 (3 adım)*

| a | b | a mod b |
|---|---|---|
| 252 | 105 | 42 |
| 105 | 42 | 21 |
| 42 | 21 | 0 |
| 21 | 0 | **b = 0: sonuç 21** |
| 1071 | 462 | 147 |
| 462 | 147 | 21 |
| 147 | 21 | 0 |
| 21 | 0 | **b = 0: sonuç 21** |

**4. Sözde kod.**

SÖZDE KOD · Öklid algoritması

```
algoritma EBOB(a, b)                ▷ ön koşul: a, b ≥ 0; ikisi birden 0 değil
    b ≠ 0 iken tekrarla             ▷ değişmez: ebob(a, b) = ebob(a₀, b₀)
        r ← a mod b                 ▷ 0 ≤ r < b
        (a, b) ← (b, r)             ▷ b kesin azalır
    döndür a                        ▷ ebob(a, 0) = a
```

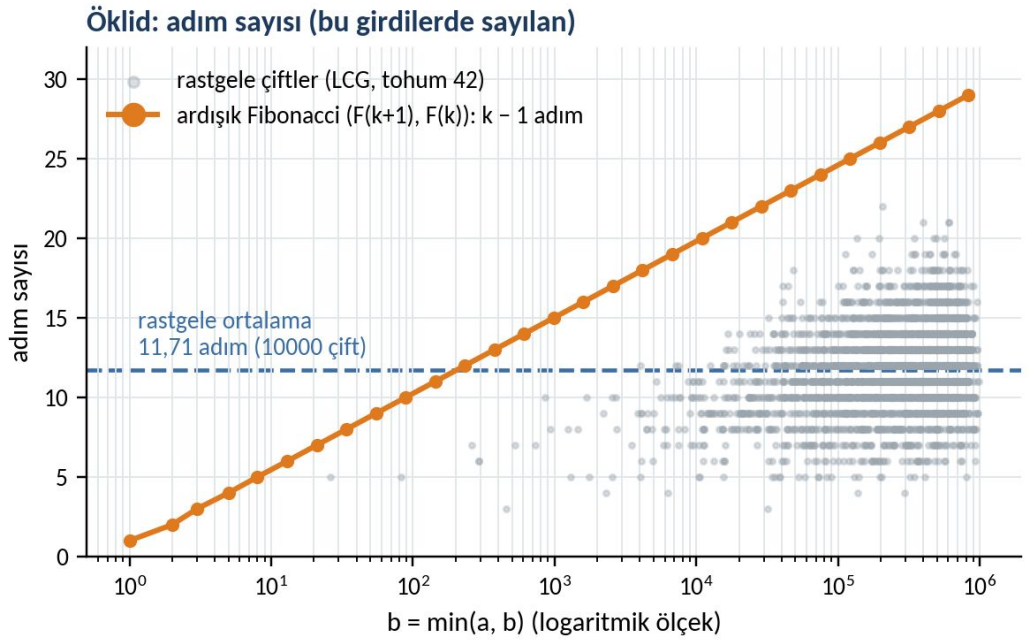
**5. Doğruluk ve sonlanma. Değişmez:** her yinelemenin başında ebob(a, b) = ebob($a_0$, $b_0$). **Başlatma:** a = $a_0$, b = $b_0$. **Koruma:** a mod b = a − ⌊a/b⌋·b olduğundan d'nin a ve b'yi bölmesi ile b ve a mod b'yi bölmesi aynı şeydir; ortak bölenler kümesi, dolayısıyla en büyüğü değişmez. **Sonlanma:** döngü b = 0'da biter; ebob(a, 0) = a. **Sonlanma kanıtı:** b ≥ 0'dır ve yerine 0 ≤ a mod b < b gelir: b **kesin azalır**; döngü en çok $b_0$ adımda biter (gerçekte çok daha hızlı).

**6. Analiz ve kimlik kartı.** En iyi durum: b = 0 ya da b, a'yı böler (**O(1)**). En kötü durum **ardışık Fibonacci sayılarıdır**: (F(k+1), F(k)) çiftinde her adım bir önceki çifte iner ve k − 1 adım sürer (Lamé teoremi; Cormen vd., 2022). Fibonacci sayıları yaklaşık $\varphi^k$ ($\varphi \approx 1{,}618$) büyüdüğünden adım sayısı **O(log min(a, b))**'dir. Programla sayılan: (8, 5) → 4; (89, 55) → 9; (10 946, 6 765) → 19; (832 040, 514 229) → 28 adım; ortak LCG ile üretilen 10 000 rastgele çiftte (a, b ∈ [1, 1000000]) ortalama 11,71, en çok 22 adım (Şekil 1.8).

**Tablo 1.10**
*Algoritma kimlik kartı: Öklid algoritması*

| ALGORİTMA KİMLİK KARTI **Öklid algoritması (EBOB)** *(Euclidean algorithm, GCD)* | |
|---|---|
| **Strateji** | Problemi küçültme: ebob(a, b) = ebob(b, a mod b) |
| **Girdi önkoşulu** | a ≥ 0, b ≥ 0 tamsayı; ikisi birden 0 değil |
| **Zaman (en iyi · ortalama · en kötü)** | O(1) · O(log min(a, b)) · O(log min(a, b)) |
| **Ek bellek** | O(1) |
| **Özellikler** | Sonlanma: b her adımda kesin azalır, 0'ın altına inemez; en kötü durum ardışık Fibonacci sayıları, ör. (832040, 514229) için 28 adım |
| **Değişmez / doğruluk** | ebob(a, b) döngü boyunca aynı kalır, çünkü ebob(a, b) = ebob(b, a mod b) |
| **Kütüphane karşılığı** | C: yok (elle yazılır) · C++: `std::gcd` (`<numeric>`, C++17) · Python: `math.gcd(a, b)` |
| **Ne zaman kullanılır** | İki tamsayının EBOB'u, kesir sadeleştirme, EKOK = a / ebob(a, b) · b |
| **Ne zaman kullanılmaz** | Asal çarpanlar gerekiyorsa (ayrı problem); negatif girdide önce mutlak değer alın (C'de `%` negatif kalan verebilir, Örnek 6) |

**Şekil 1.8**
*Öklid'in adım sayısı: ardışık Fibonacci çiftleri (en kötü durum, k − 1 adım) ve rastgele çiftler; x logaritmik ölçekte, adım sayısı log b ile doğrusal büyür*



**7. C gerçekleştirimi.** Dosyadaki yorum sözleşmeyi, değişmezi ve azalan niceliği yazar; `adim_sayisi` yalnızca deney için sayaçtır.

C **H01_05_oklid_ebob.c · satır 22–32** gcc -std=c17 -Wall -Wextra -pedantic -O2 H01_05_oklid_ebob.c -o H01_05_oklid_ebob

```c
int ebob(int a, int b)
{
    adim_sayisi = 0;
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
        adim_sayisi++;
    }
    return a;
}
```

ÇIKTI · ÜÇ DİLDE AYNI

```
$ ./H01_05_oklid_ebob          # ikinci bölüm (ilki izleme tablosundadır)
        a         b     ebob  adım
      252       105       21     3
     1071       462       21     3
       55        89        1    10
       17         0       17     0
        0         9        9     1
  1000000    999999        1     2
       89        55        1     9
   832040    514229        1    28
```

**8. Üç dil karşılaştırması.** C++ gövdesi C ile aynıdır (Ek A). Python'un çoklu ataması `a, b = b, a % b` geçici değişkeni kaldırır: sağ taraf önce bütünüyle hesaplanır, sonra iki atama birlikte yapılır; sayaç modül düzeyinde olduğundan `global` bildirimi gerekir.

Python **H01_05_oklid_ebob.py · satır 21–28**

```python
def ebob(a, b):
    """Öklid algoritmasıyla ebob(a, b)."""
    global adim_sayisi
    adim_sayisi = 0
    while b != 0:
        a, b = b, a % b
        adim_sayisi += 1
    return a
```

**9. Test.** Sınırlar çıktı tablosundadır: (17, 0) → 0 adım; (0, 9) → 1 adım; (55, 89) → yer değiştirme; (1000000, 999999) → ardışık sayılar aralarında asaldır. Kâhin: kaba kuvvet (min(a, b)'den aşağı ilk ortak bölen) ya da `std::gcd`, `math.gcd`. Negatif girdi ön koşulu çiğner: C'de `%` negatif kalan verebilir (Örnek 6).

**10. Deney.** Ölçülen büyüklük süre değil **adım sayısıdır**: Şekil 1.8'de x logaritmik olduğundan Fibonacci çiftleri bir doğru üzerindedir.

**11. Varyantlar ve sonraki basamak.** Özyinelemeli yazım `ebob(a, b) = (b == 0) ? a : ebob(b, a % b)` (H3); EKOK: ekok(a, b) = a / ebob(a, b) · b, önce bölmek taşmayı önler (Alıştırma 1.11); genişletilmiş Öklid (Merak Kutusu).

# 5. Çözümlü Örnekler: Merdivenin Kalan Basamakları

Örnek 1, 2, 4 ve 5 Bölüm 4'te, Örnek 3'ün değişmezi Bölüm 3.5'te, Örnek 8 Bölüm 3.11'de işlendi. Bu bölüm kalan basamakları C kesitleri ve gerçek çıktılarla verir; C++ ve Python karşılıkları Ek A'dadır. Başlıklarda düzey (★) ve çıktı türü (=: üç dilde aynı, Δ: bilinçli dil farkı) yazılıdır.

### Örnek 3 · Toplam: döngü ve formül (★, =)

Gauss formülü n(n + 1)/2 döngüyle aynı sonucu verir; ama çarpım hangi türde yapılırsa o türün sınırına tabidir. n = 65536 için sonuç (2147516416) 32 bitlik `int`'in en büyük değerini (2147483647) aşar; çarpım `long long`'da yapılır:

C **H01_03_toplam_1_n.c · satır 29–32**

```c
29 long long toplam_formul(int n)
30 {
31     return (long long)n * (n + 1) / 2;
32 }
```

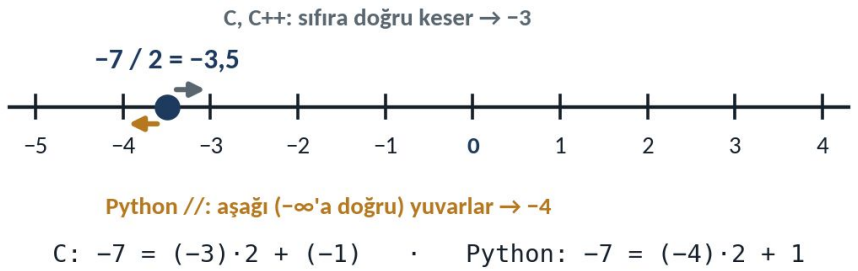
ÇIKTI

```
$ ./H01_03_toplam_1_n          # çıktının ikinci bölümü
       n            döngü           formül  eşit
       0                0                0  evet
       1                1                1  evet
      10               55               55  evet
     100             5050             5050  evet
   65536       2147516416       2147516416  evet
 1000000     500000500000     500000500000  evet
```

### Örnek 6 · −7 / 2 ve −7 % 2 (★★, Δ)

C (C99'dan beri) ve C++ (C++11'den beri) tamsayı bölmesini **sıfıra doğru keser** (truncation toward zero), kalan **bölünenin** işaretini alır; Python'un `//` işleci **taban bölmesi** (floor division) yapar, yani **aşağı** (−∞'a doğru) yuvarlar ve `%` **bölenin** işaretini alır (Şekil 1.9). Üç dilde de (a / b)·b + a % b = a özdeşliği korunur: bölüm farklıysa kalan da farklıdır.

**Şekil 1.9**
*−7 / 2 = −3,5 sayı doğrusunda: C ve C++ sıfıra doğru keser (−3), Python // aşağı yuvarlar (−4)*



**H01_06_bolme_kalan.c · satır 19–30**

```c
    for (int i = 0; i < 4; i++) {
        int a = a_deger[i];
        int b = b_deger[i];
        int q = a / b;      /* bölüm: sıfıra doğru kesme */
        int r = a % b;      /* kalan: işareti a'nınki    */
        printf("%4d %4d %7d %7d %13d\n", a, b, q, r, q * b + r);
    }
    /* Yaygın tuzak: x % 2 == 1 negatif tek sayılarda yanlıştır. */
    printf("-7 tek mi? (-7 %% 2 == 1): %s\n",
           (-7 % 2 == 1) ? "evet" : "hayır");
    printf("-7 tek mi? (-7 %% 2 != 0): %s\n",
           (-7 % 2 != 0) ? "evet" : "hayır");
```

**ÇIKTI · C (C++ AYNI)**

```
$ ./H01_06_bolme_kalan
C: / sıfıra doğru keser, % bölünenin işaretini alır
   a    b   a / b   a % b   (a / b) * b + a % b
   7    2       3       1             7
  -7    2      -3      -1            -7
   7   -2      -3       1             7
  -7   -2       3      -1            -7
-7 tek mi? (-7 % 2 == 1): hayır
-7 tek mi? (-7 % 2 != 0): evet
```

Python'da tablo satırı "-7 2 -4 1 -7", tuzak satırı "-7 tek mi? (-7 % 2 == 1): evet" olur (Ek A). Yaygın hata: `== 1` C ve C++'ta negatif tek sayıyı kaçırır; taşınabilir test `x % 2 != 0`'dır.

### Örnek 7 · 13!: uint32_t sarması ve keyfi duyarlıklı tamsayı (★★, Δ)

Bir sonuç türünün aralığına sığmadığında **taşma** (overflow) olur. C ve C++'ta **işaretsiz** tamsayılar $2^N$ modunda sarar: standardın tanımladığı davranıştır (ISO/IEC, 2018, §6.2.5). **İşaretli** taşma ise **tanımsız davranıştır** (§6.5): derleyici taşmanın hiç olmayacağını varsayabilir (HA4). Program bu yüzden yalnızca işaretsiz türler kullanır:

**H01_07_faktoriyel_tasma.c · satır 27–34**

```c
    for (uint32_t n = 1; n <= 13; n++) {
        f32 *= n;       /* 2^32 modunda: tanımlı sarma */
        f64 *= n;
        printf("%2" PRIu32 " %13" PRIu32 " %21" PRIu64 "%s\n", n, f32, f64,
               (uint64_t)f32 == f64 ? "" : "  <- uint32_t sardı");
    }
    printf("13! mod 2^32 = %" PRIu64 " - %" PRIu64 " = %" PRIu32 "\n",
           f64, (uint64_t)1 << 32, f32);
```

ÇIKTI · C (C++ AYNI)

```
$ ./H01_07_faktoriyel_tasma          # 1!–10! satırları atlandı
C: işaretsiz sabit genişlikli tamsayılar (2^N modunda sarar)
 n      uint32_t              uint64_t
11      39916800              39916800
12     479001600             479001600
13    1932053504            6227020800  <- uint32_t sardı
13! mod 2^32 = 6227020800 - 4294967296 = 1932053504
20! (uint64_t) = 2432902008176640000
21! (uint64_t) = 14197454024290336768  <- uint64_t de sardı
```

Python'un `int`'i keyfi duyarlıklıdır: "21! = 51090942171709440000 (sarma yok: keyfi duyarlık)", "21! için gereken bit sayısı: 66". Bedeli hızdır: her işlem değişken uzunluklu bir nesne işlemidir. Sarmayı Python'da taklit etmek için `& 0xFFFFFFFF` maskesi kullanılır.

### Örnek 9 · Hatalı en büyüğü testle yakalamak (★★, =)

`en_buyuk_hatali` m = 0 ile başlar (dosya **kasıtlı hata içerir**); doğru sürüm (Örnek 1) kâhindir. Üç test türü aynı hataya çok farklı sonuç verir:

C **H01_09_hatali_en_buyuk.c · satır 32–41**

```c
int en_buyuk_hatali(const int a[], size_t n)
{
    int m = 0;                    /* HATA: m = a[0] olmalıydı */
    for (size_t i = 0; i < n; i++) {
        if (a[i] > m) {
            m = a[i];
        }
    }
    return m;
}
```

ÇIKTI · ÜÇ DİLDE AYNI

```
$ ./H01_09_hatali_en_buyuk
1) Örnek test (tipik girdi):
  [3, 9, 4]: beklenen 9, bulunan 9 -> geçti (tipik)
2) Sınır değer testleri:
  [5]: beklenen 5, bulunan 5 -> geçti (tek eleman)
  [-5]: beklenen -5, bulunan 0 -> YAKALANDI (tek negatif eleman)
  [-8, -3, -12]: beklenen -3, bulunan 0 -> YAKALANDI (tamamı negatif)
  [9, 1, 2]: beklenen 9, bulunan 9 -> geçti (en büyük başta)
  [0, -1]: beklenen 0, bulunan 0 -> geçti (en büyük 0)
3) Rastgele test (ortak LCG, tohum 42, n = 1..8):
  ilk karşı örnek (deneme 17): [-77] -> beklenen -77, bulunan 0
  değerler [-100, 100]: 1000 denemede 128 uyuşmazlık
  değerler [0, 100]: 1000 denemede 0 uyuşmazlık (hata görünmedi!)
```

Tipik test hatayı göstermez; sınır değerleri (tek negatif eleman, tamamı negatif) hemen yakalar. Rastgele test [−100, 100]'de karşı örnek bulur, ama [0, 100]'de **hiç** bulmaz: girdi dağılımı hatanın tetiklendiği bölgeyi kapsamıyorsa rastgele test de kördür. Ortak LCG sayesinde aynı karşı örnek üç dilde aynı denemede çıkar.

### Örnek 10 · İkinci en büyük ve kâhinle karşılaştırmalı test (★★★, =)

Tek geçişte iki rekor tutulur: b1 ≥ b2, a[0..i−1]'in en büyük iki elemanı (yinelenenler ayrı sayılır). a[i] > b1 ise iki rekor kayar (`b2 = b1; b1 = a[i];`); b2 < a[i] ≤ b1 ise yalnız b2 değişir (`else if (a[i] > b2)` dalı). Kâhin **farklı bir yoldan** hesaplar: kopyayı `qsort` ile sıralar ve sondan ikinciyi alır (Θ(n log n); kütüphane yalnızca kâhinde serbesttir). Test döngüsü aynı rastgele girdiyi ikisine verir ve ilk uyuşmazlıkta girdiyi yazdırır. Testi sınamak için `else if` dalı silinmiş **kasıtlı hatalı** sürüm de teste sokulur (a = [5, 1, 8, 3, 8, 6] için 5 döndürür):

C **H01_10_ikinci_en_buyuk.c · satır 108–121**

```c
    for (int t = 1; t <= deneme; t++) {
        size_t n = (size_t)rastgele_aralik(2, MAKS_N);
        rastgele_dizi(a, n, alt, ust);
        int beklenen = kahin(a, n);
        int bulunan = f(a, n);
        if (beklenen != bulunan) {
            uyusmazlik++;
            if (uyusmazlik == 1) {
                printf("  ilk uyuşmazlık (deneme %d): ", t);
                dizi_yaz(a, n);
                printf(" -> kâhin %d, bulunan %d\n", beklenen, bulunan);
            }
        }
    }
```

ÇIKTI · ÜÇ DİLDE AYNI

```
$ ./H01_10_ikinci_en_buyuk          # 1) sınır durumları atlandı
2) Kâhinle rastgele karşılaştırmalı test (tohum 2027):
  ikinci_en_buyuk, değerler [-5, 5]: 2000 deneme, 0 uyuşmazlık
  ikinci_en_buyuk, değerler [-1000, 1000]: 2000 deneme, 0 uyuşmazlık
3) Testi sına: kasıtlı hatalı sürüm aynı testte
  ilk uyuşmazlık (deneme 2): [4, -4, 1, 1, 2, -1, -4] -> kâhin 2, bulunan -4
  ikinci_en_buyuk_hatali, değerler [-5, 5]: 2000 deneme, 978 uyuşmazlık
4) Stres testi: n = 100000, değerler [-10^6, 10^6], tohum 7
  hızlı 999953, kâhin 999953 -> eşleşti
```

Atlanan ilk bölümdeki 7 sınır durumunun (n = 2, eşitlik, en büyük başta/sonda) hepsinde sonuç kâhinle aynıdır. Dar aralık ([−5, 5]) bilinçli seçildi: çok sayıda eşitlik üretir ve eşitlik dalının hatasını hemen ortaya çıkarır.

### Örnek 11 · Süre ölçümü: üç saat, üç dil (★★, =)

Aynı toplam üç yolla ölçülür: duvar saati (`timespec_get`), işlemci zamanı (`clock`) ve ortak `zamanlayici` modülünün `olc_ortanca`'sı (1 ısınma + 5 tekrar). stdout'a yalnızca deterministik değerler ("toplam = 49475615"), süreler stderr'e yazılır:

**H01_11_sure_olcumu.c · satır 68–74**

```c
    /* 1) Duvar saati: timespec_get (C11), nanosaniye çözünürlüklü */
    struct timespec t0, t1;
    timespec_get(&t0, TIME_UTC);
    long long s1 = toplam(a, (size_t)n);
    timespec_get(&t1, TIME_UTC);
    double duvar = (double)(t1.tv_sec - t0.tv_sec)
                 + (double)(t1.tv_nsec - t0.tv_nsec) * 1e-9;
```

STDERR · BU MAKİNEDE TEK BİR ÇALIŞTIRMA (C++ SATIRLARI ATLANDI)

```
$ ./H01_11_sure_olcumu > /dev/null        # C, -O2
timespec_get (duvar saati)      :      0.704 ms
clock (işlemci zamanı)          :      0.622 ms
olc_ortanca (1 ısınma + 5, ort.):      0.459 ms
$ python3 H01_11_sure_olcumu.py > /dev/null
perf_counter (monotonik saat)   :     40.544 ms
process_time (işlemci zamanı)   :     40.643 ms
olc_ortanca (1 ısınma + 5, ort.):     41.371 ms
```

Bu çalıştırmada ısınmasız ilk ölçüm (duvar saati) ile 1 ısınma + 5 tekrarın ortancası: C'de 0,704 ms'ye karşı 0,459 ms (%53 fazla); Python'da belirgin fark yok (40,5 ms'ye karşı 41,4 ms). İlk ölçümün ek maliyeti (soğuk önbellek, ilk sayfa erişimleri) 0,5 ms'lik C ölçümünde büyük pay tutar, 41 ms'lik Python ölçümünde gürültüye karışır. C ve C++ aynı mertebede, Python iki mertebe yukarıdadır (Bölüm 6).

### Örnek 12 · Ölçüm tuzağı: -O2'de kaybolan döngü (★★★, =)

İki fonksiyon aynı döngüyü çalıştırır: `kullanilmayan_dongu` sonucu atar, `kullanilan_dongu` ise son satırda `sonuc_yeri = s;` ile dış bir değişkene yazar (Bölüm 3.9). stdout üç dilde aynıdır; fark yalnızca süredir ve stderr'dedir:

**H01_12_olcum_tuzagi.c · satır 28–36**

```c
void kullanilmayan_dongu(void *veri)
{
    const Dizi *d = veri;
    long long s = 0;
    for (size_t i = 0; i < d->n; i++) {
        s += d->a[i];
    }
    (void)s;                /* yalnızca derleyici uyarısını susturur */
}
```

**Tablo 1.11**
*Örnek 12, $n = 10^6$: çalıştırma ortancaları (bu makinede ölçülen; deney/tuzak_olc.py)*

| Yapılandırma | çalıştırma | sonucu kullanılmayan | sonucu kullanılan | kullanılan, ns/eleman |
|---|---|---|---|---|
| C -O0 | 5 | 2,17 ms | 2,20 ms | 2,20 |
| C -O2 | 5 | 26 ns (silindi) | 0,462 ms | 0,46 |
| C++ -O0 | 5 | 4,63 ms | 4,66 ms | 4,66 |
| C++ -O2 | 5 | 27 ns (silindi) | 0,457 ms | 0,46 |

| Yapılandırma | çalıştırma | sonucu kullanılmayan | sonucu kullanılan | kullanılan, ns/eleman |
|---|---|---|---|---|
| Python (CPython) | 3 | 38,9 ms | 41,0 ms | 41,00 |

-O2'de sonucu kullanılmayan döngünün süresi C'de 26 ns, C++'ta 27 ns'dir: n = $10^6$ elemanı gerçekten toplamak ≈ 0,46 ms sürerdi. gcc döngüyü silmiştir (yalnızca `ret`); ölçülen, iki saat çağrısının kendi maliyetidir. -O0'da iki döngü aynı sürer; C++ -O0'ın yavaşlığı, -O0'da `std::vector` erişiminin satır içine alınmayan çağrı olmasındandır. CPython ölü kodu silmez (38,9 ms ve 41,0 ms).

### Örnek 13 · Deney raporu: n'e karşı süre tablosu ve CSV (★★★, =)

n = $10^3$, $10^4$, $10^5$ (`--nmax` ile $10^8$'e kadar) için deterministik tablo stdout'a, ortanca süre ve T(n)/T(n/10) stderr'e, `--csv dosya` ile CSV'ye yazılır. `derleme` sütunu C/C++'ta `__OPTIMIZE__` makrosundan dolar: makro -O1 ve üstünde tanımlı olduğundan -O2 derlemesi `O1+`, -O0 derlemesi `O0` yazar.

C `H01_13_deney_raporu.c · satır 102–112`

```c
        double t = olc_ortanca(toplam_gorevi, &g, TEKRAR);
        printf("%10ld %14lld %10.4f\n", n, g.sonuc,
               (double)g.sonuc / (double)n);
        fprintf(stderr, "n = %9ld: ortanca %10.4f ms", n, t * 1e3);
        if (onceki > 0.0) {
            fprintf(stderr, "   T(n)/T(n/10) = %.2f", t / onceki);
        }
        fprintf(stderr, "\n");
        if (csv != NULL) {
            fprintf(csv, "%ld;C;%s;%d;%.6f;%lld\n", n, DERLEME, TEKRAR,
                    t * 1e3, g.sonuc);
```

stdout üç dilde aynıdır (son satırı "100000 4933249 49.3325"). CSV dosyası şöyle başlar (bu makinede, C -O2, `--nmax 1000000`); Atölye 3'te bu dosyalar birleştirilir, web sayfasındaki ölçüm laboratuvarı da aynı biçimdeki dosyayı okuyup grafiğini çizer:

CSV · C_O2.CSV (İLK SATIRLAR)

```
$ ./H01_13_deney_raporu --nmax 1000000 --csv c_O2.csv
n;dil;derleme;tekrar;ortanca_ms;toplam
1000;C;O1+;5;0.000443;49681
10000;C;O1+;5;0.004257;490479
…
```

# 6. Haftanın Deneyi: Aynı Algoritma, Üç Dil, İki Derleme Düzeyi

> HAFTANIN DENEYİ · **Soru**
>
> Aynı doğrusal toplam algoritmasının C (-O0 ve -O2), C++ (-O2) ve Python sürümlerinin n = $10^5$…$10^7$ için süreleri; tek ölçüm, 5 tekrarın ortalaması ve 5 tekrarın ortancası arasındaki tutarlılık farkı. Kuram: algoritma Θ(n)'dir; n on katına çıkınca süre de ≈ on katına çıkmalı, log–log eğim 1 olmalıdır.

### Ölçüm düzeni ve ortam

Programlar `kodlar/deney/` altındadır (sürücü `deney_calistir.py`). Her **oturum** ayrı bir süreçtir: ortak LCG ile veri (tohum 42, değerler 0..99; üç dilde aynı dizi), **1 soğuk ölçüm** (ısınmasız "tek ölçüm") ve

**5 tekrar**; “ortalama” ve “ortanca” bu 5 tekrardan hesaplanır. Yapılandırmalar dönüşümlü çalıştırılır; oturumlar arası yayılımı %20’yi aşan yapılandırmada ek oturum yapılır. Tablodaki değer **oturum ortancalarının ortancasıdır**; her işin sonucu denetlenir ve yazdırılır.

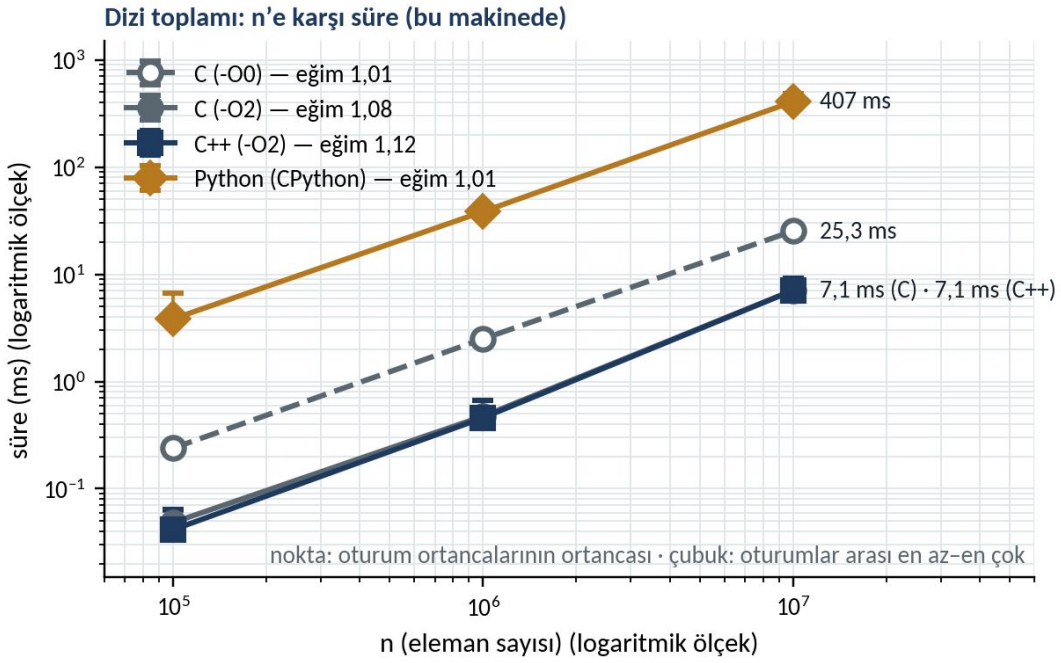
| | |
|---|---|
| **İşlemci** | Intel(R) Xeon(R) Processor @ 2.80GHz · 2 çekirdek · KVM sanal makine |
| **Önbellek** | L1d 64 KiB (2 instances) · L2 2 MiB (2 instances) · L3 33 MiB (1 instance) |
| **Derleyiciler** | gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0 · g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0 · Python 3.11.15 (CPython) |
| **Bayraklar** | C: -std=c17 -Wall -Wextra -pedantic -O0 \| -O2 · C++: -std=c++17 -Wall -Wextra -pedantic -O2 |
| **Tarih ve yük** | 2026-10-13 02:56 – 02:58 (deney süresi 107 s) · başta 1.40, sonda 1.39 — 2 çekirdekli sanal makine; başka süreçler çalışıyordu |

## Sonuçlar

**Tablo 1.12**
*Dizi toplamı: oturum ortancalarının ortancası ve eleman başına süre (bu makinede ölçülen)*

| n | C -O0 | C -O2 | C++ -O2 | Python |
|---|---|---|---|---|
| 100 000 | 0,239 ms (2,39 ns/el.) | 0,048 ms (0,48 ns/el.) | 0,041 ms (0,41 ns/el.) | 3,83 ms (38,33 ns/el.) |
| 1 000 000 | 2,50 ms (2,50 ns/el.) | 0,474 ms (0,47 ns/el.) | 0,459 ms (0,46 ns/el.) | 38,4 ms (38,40 ns/el.) |
| 10 000 000 | 25,3 ms (2,53 ns/el.) | 7,09 ms (0,71 ns/el.) | 7,07 ms (0,71 ns/el.) | 407 ms (40,73 ns/el.) |

**Şekil 1.10**
*n’e karşı süre, dört yapılandırma (log–log; nokta: oturum ortancalarının ortancası, çubuk: oturumlar arası en az–en çok)*



**Tablo 1.13**
*Oran testi ve log–log eğim (kuram: oran 10, eğim 1)*

| Yapılandırma | $T(10^6)/T(10^5)$ | $T(10^7)/T(10^6)$ | log–log eğim |
|---|---|---|---|
| C -O0 | 10,45 | 10,16 | 1,01 |
| C -O2 | 9,81 | 14,94 | 1,08 |
| C++ -O2 | 11,08 | 15,40 | 1,12 |
| Python | 10,02 | 10,61 | 1,01 |

## Kuramla karşılaştırma ve yorum

- **Eğim ≈ 1:** dört yapılandırmada log–log eğim 1,01, 1,08, 1,12, 1,01: süre n ile doğrusal büyür. Dil ve derleme düzeyi **sabit çarpanı** değiştirir, büyüme hızını değil.
- **Oranlar:** C -O0'da T(10n)/T(n) = 10,45, 10,16, Python'da 10,02, 10,61 (kuram 10). -O2'de C 9,81, 14,94, C++ 11,08, 15,40: $10^6 \rightarrow 10^7$ adımında oran 10'u aşar, çünkü 40 MB'lık dizi 33 MiB'lık L3 önbelleğe sığmaz ve veri ana bellekten gelir (C -O2'de eleman başına 0,48 → 0,47 → 0,71 ns). -O0 ve Python'da eleman başına iş zaten yavaş olduğundan bu bellek darboğazı görünmez. $10^5$'teki ≈ 48 µs'lik -O2 ölçümleri gürültüye en duyarlı ölçümlerdir.
- **Sabit çarpanlar:** Python / C -O2 oranı 79,3×, 81,0×, 57,5×: bir ila iki büyüklük mertebesi (Bölüm 3.10). -O0 → -O2 kazancı (4,9×, 5,3×, 3,6×) -O0'da s ve i'nin her yinelemede belleğe yazılıp okunmasının kalkmasındandır.
- **C ve C++ farkı yorumlanmamalı:** iki döngü de aynı 5 komutluk makine koduna derlenir; küçük farklar gürültüdür.

## Tek ölçüm, 5 tekrarın ortalaması, 5 tekrarın ortancası

**Tablo 1.14**
*Oturumlar arası yayılım, (en çok − en az) / ortanca; son sütun: tek ölçümün aynı oturumun ortancasından tipik sapması*

| Yapılandırma, $n = 10^6$ | tek ölçüm | 5 tekrar ortalaması | 5 tekrar ortancası | tek − ortanca (oturum içi, ortanca) |
|---|---|---|---|---|
| C -O0 | %7 | %7 | %3 | %1,7 |
| C -O2 | %40 | %56 | %45 | %42,8 |
| C++ -O2 | %51 | %58 | %53 | %16,9 |
| Python | %11 | %20 | %16 | %−0,8 |

**Bulgular (veriden).** (1) Isınmasız tek ölçüm -O2'de belirgin biçimde yüksektir: $n = 10^6$'da aynı oturumun ortancasından tipik %43 (C), %17 (C++), en çok %80 büyük; -O0'da ve Python'da fark küçüktür (%2, %−1), çünkü ilk ölçümün ek maliyeti uzun süren işin içinde kaybolur. (2) Ortalama aykırı değere duyarlıdır: tek bir gecikmiş tekrar ortalamayı ortancanın en çok %63 üstüne çekti (C -O2, $10^5$). (3) Ortanca yük altındaki bir **oturumu** düzeltemez (makine 2 çekirdekliydi ve başka süreçler çalışıyordu): oturumu tekrarlamak ve en az–en çok aralığını raporlamak gerekir. (4) Python içinde yazım farkı ($n = 10^6$): indisli döngü 40,3, `for x in a` 25,8, C ile yazılmış yerleşik `sum(a)` 6,1 ns/eleman; `sum` algoritmanın yerine geçmez ama yorumlayıcı döngüsünden çıkmanın sabit çarpanı küçülttüğünü gösterir.

**Sınırlılıklar.** Tek bir 2 çekirdekli KVM sanal makinesi; ölçüm sırasında başka süreçler çalışıyordu: sayılar "bu makinede ölçülen"dir (kendi makinenizde `python3 deney_calistir.py`, ≈ 2 dakika). Oturum sayısı 7 ya da 11 (tutarsız yapılandırmalarda ek oturum); yayılım betimsel verildi, istatistiksel test yapılmadı. C'nin `zaman_sn()`'i `timespec_get(TIME_UTC)` kullanır ve monotonik değildir (C++ ve Python monotonik saat kullanır); kısa ölçümlerde pratik etkisi yoktur. Sabit çarpanlar işlemciye ve derleyici/CPython sürümüne bağlıdır; eğimin ≈ 1 olması makineden bağımsız beklenir.

# 7. Hata Avcısı

Altı hata, altı ders (Tablo 1.15). Kasıtlı hatalı dosyalar `kodlar/hata_avcisi/` altındadır (başlıklarında "BU DOSYA KASITLI HATA İÇERİR" yazar); düzeltilmiş sürümler `_duzeltilmis` ekiyle, gerçek çıktılar `hata_avcisi/ciktilar/` altındadır. Kasıtlı hatalı C dosyaları da `-Wall -Wextra -pedantic` ile **uyarısız** derlenir: bu hataların hiçbirini derleyici yakalamaz.

**Tablo 1.15**
*Hata Avcısı: hata, belirti, neden ve çözüm*

| Hata | Belirti (gerçek çıktı) | Neden | Çözüm |
|---|---|---|---|
| HA1 · En büyükte başlangıç değeri 0 | "Sıcaklıklar : en yüksek 0 °C"; puanlarda doğru ("Puanlar : en yüksek 72") | 0 dizinin elemanı değil; değişmez başlangıçta kurulmamış | m = a[0], döngü i = 1'den; test: tamamı negatif, tek eleman |
| HA2 · Gövde sonunda doğru olan ifade değişmez sanılmış | İlk yinelemede "Assertion … failed.", Aborted (çıkış kodu 134) | s = k(k+1)/2 yalnızca `s += k`'dan sonra doğru | Döngü başında s = (k−1)k/2'yi denetle |
| HA3 · Tek ölçümle ve -O0 ile genelleme | Ayrı çalıştırmalarda farklı süreler; -O0 ile -O2 arasında kat kat fark | Soğuk ölçüm, arka plan yükü; -O0 üretimde kullanılan kod değil | -O2, ısınma + tekrar + ortanca; en az–en çok; "bu makinede" |
| HA4 · İşaretli taşmanın "başa sarar" sanılması | -O0 ve -O2 farklı yanıt; UBSan: "signed integer overflow" | İşaretli taşma tanımsız davranış; derleyici x + 1 < x'i yanlış sayabilir | İşlemden önce sınırla karşılaştır; işaretsiz tür; UBSan |
| HA5 · Python `//` = C `/` sanmak | "dk_sn(-125) = (-3, 55), C'deki = (-2, -5) -> UYUŞMADI" | C sıfıra doğru keser; Python aşağı yuvarlar | Niyeti açıkça yaz (`bol_sifira_dogru`) ya da anlamı bilinçli seç |
| HA6 · rand() ile tekrarlanamayan deney | İki çalıştırmada farklı veri ve toplam | srand(time(NULL)); rand() algoritması kitaplığa bağlı | Ortak LCG + sabit tohum; tohumu kaydet |

### HA2 · Yanlış yerde doğru olan "değişmez"

Döngü başına konan `assert(s == (long long)k * (k + 1) / 2);` ilk yinelemede patlar:

```
ÇIKTI
$ ./H01_HA2_yanlis_degismez
toplam(5) hesaplanıyor...
H01_HA2_yanlis_degismez: H01_HA2_yanlis_degismez.c:20: toplam: Assertion `s == (long
long)k * (k + 1) / 2' failed.
Aborted
[çıkış kodu: 134]
```

k = 1, s = 0'da 1·2/2 = 1 ≠ 0: iddia döngü başında yanlıştır. Düzeltme döngü başında (k−1)k/2'yi denetler; sonlanmada k = n + 1 olur ve iddia son koşulu verir (`…_duzeltilmis.c`).

## HA4 · İşaretli taşma: tanımsız davranış

> **HATA AVCISI · Bu örnek tanımsız davranışı bilerek çalıştırır**
>
> x = INT_MAX için iki fonksiyon da işaretli taşma yapar: C standardına göre bu **tanımsız davranıştır** ve programın ne yapacağı hakkında hiçbir güvence yoktur. Aşağıdaki çıktılar **bu derleyicide (gcc 13.3, x86-64) ve bu çalıştırmada gözlenen** davranıştır; başka derleyici, sürüm ya da bayrakla farklı olabilir. Amaç, "taşarsa negatife sarar" varsayımına dayanan kodun güvenilmez olduğunu görmektir; böyle kod yazılmaz.

C **hata_avcisi/H01_HA4_isaretli_tasma.c · satır 16–26**

```c
int tasar_mi_1(int x)
{
    return x + 1 < x;           /* HATA: x == INT_MAX ise tanımsız */
}

/* Aynı varsayım, toplama ayrı bir deyimde. */
int tasar_mi_2(int x)
{
    int y = x + 1;              /* HATA: x == INT_MAX ise tanımsız */
    return y < x;
}
```

**ÇIKTI · -O0, -O2 VE UBSAN (BU DERLEYİCİDE GÖZLENEN)**

```
$ ./H01_HA4_isaretli_tasma_O0
x = 2147483647
1. sürüm: x + 1 taşar mı? hayır
2. sürüm: x + 1 taşar mı? evet
[çıkış kodu: 0]
$ ./H01_HA4_isaretli_tasma_O2
x = 2147483647
1. sürüm: x + 1 taşar mı? hayır
2. sürüm: x + 1 taşar mı? hayır
[çıkış kodu: 0]
$ ./H01_HA4_isaretli_tasma_ubsan
H01_HA4_isaretli_tasma.c:18:14: runtime error: signed integer overflow: 2147483647 +
1 cannot be represented in type 'int'
[çıkış kodu: 1]
```

Programcı iki sorguda da "evet" bekler. gcc 1. sürümdeki `x + 1 < x`'i -O0'da bile sabit 0'a indirdi; 2. sürüm -O0'da "evet", -O2'de "hayır" dedi. `-fsanitize=undefined` taşmayı çalışma anında raporlar. Doğru yol işlemden **önce** sınırla karşılaştırmaktır (`y > 0 && x > INT_MAX - y`; `H01_HA4_isaretli_tasma_duzeltilmis.c`); GCC/Clang'da `__builtin_add_overflow`, C23'te `ckd_add` aynı işi görür.

## HA5 · Python'a "birebir" çeviri

Python **hata_avcisi/H01_HA5_taban_bolme.py · satır 19–21**

```python
def dk_sn(fark):
    """Saniye cinsinden farkı (dakika, saniye) olarak döndürür."""
    return fark // 60, fark % 60     # HATA: C'nin / ve % kuralı değil
```

ÇIKTI

```
$ python3 H01_HA5_taban_bolme.py
dk_sn(125) = (2, 5), C'deki = (2, 5) -> geçti
dk_sn(60) = (1, 0), C'deki = (1, 0) -> geçti
dk_sn(0) = (0, 0), C'deki = (0, 0) -> geçti
dk_sn(-125) = (-3, 55), C'deki = (-2, -5) -> UYUŞMADI
dk_sn(-30) = (-1, 30), C'deki = (0, -30) -> UYUŞMADI
5 testten 2 tanesi uyuşmadı
[çıkış kodu: 1]
```

C'nin sıfıra doğru kesen bölmesi Python'da mutlak değerlerle bölüp işareti sonradan vererek yazılır; kalan a − b·(a / b) ile bulunur (`bol_sifira_dogru`, `H01_HA5_taban_bolme_duzeltilmis.py`).

### HA1, HA3 ve HA6 · kısaca

**HA1**'in kodu Örnek 9'daki `en_buyuk_hatali` ile aynı hatayı taşır (`int m = 0;`): kış sabahı −8, −3, −12, −5 °C ölçen istasyonlar için "en yüksek 0 °C" mantıksızdır, tipik puanlarda hata görünmez; düzeltilmiş sürüm "Sıcaklıklar : en yüksek -3 °C" yazar. **HA3**'te n derleme anında sabit (100 000) olduğundan gcc -O2 döngüyü SSE2 ile vektörleştirdi; ayrı süreçlerdeki on çalıştırmada -O2 tek ölçümün en çok/en az oranı 1,95, -O0 tek ölçüm ortancasının -O2 ısınma + tekrar ortancasına oranı 13,32 çıktı: tek bir derleme ve bağlamdan genelleme yapılmaz. **HA6**'da `srand(time(NULL))` iki çalıştırmada farklı veri üretti ("100000 değerin toplamı: 4944984", "100000 değerin toplamı: 4947312"); sabit tohum bile yalnızca **bu kitaplıkta** aynı diziyi verir, çünkü standart `rand()`'ın algoritmasını belirlemez (`RAND_MAX` Microsoft C çalışma zamanında 32767, bu makinedeki glibc'de 2147483647). Düzeltilmiş sürüm ortak LCG ve sabit tohum kullanır.

## 8. Dil Penceresi: C · C++ · Python

Sözdizimi benzer ama anlamı farklı yapılar en sinsi hatalara yol açar: kod çevrilirken derleyici ya da yorumlayıcı hiçbir şey söylemez (Tshukudu & Cutts, 2020). Tek satırlık gösterim `kavram/H01_K06_bolme_tek_satir.*` dosyalarındadır.

C'de `printf("%d %d\n", -7 / 2, -7 % 2);`, Python'da `print(-7 // 2, -7 % 2)`. C ve C++ "-3 -1", Python "-4 1" yazar. Sıfıra doğru kesme C'de C99'dan, C++'ta C++11'den beri standarttır (ISO/IEC, 2017, 2018); Python'un `//` ve `%` işleçleri taban bölmesi kuralını izler; C'nin kalanı gerekiyorsa `math.fmod` kullanılabilir, ama sonucu `float`'tır ve çok büyük tamsayılarda kesinlik yitirir, bu yüzden tamsayılar için HA5'teki gibi bir yardımcı fonksiyon daha güvenlidir (Python Software Foundation, t.y.). Haftanın farkları Tablo 1.16'da, kütüphane karşılıkları Tablo 1.17'dedir; kütüphaneler yalnızca kâhin olarak ya da gerçek projede bilinçli seçim olarak kullanılır.

**Tablo 1.16**
*Bu haftanın dil farkları (ders örneklerinde rastgelelik için ortak LCG kullanılır)*

| Öğe | C | C++ | Python |
|---|---|---|---|
| Tamsayı bölmesi | `-7 / 2 → -3` (sıfıra doğru; C99) | `-7 / 2 → -3` (C++11) | `-7 // 2 → -4` (aşağı); `7 / 2 → 3.5` |
| Kalan | `-7 % 2 → -1` (bölünenin işareti) | `-7 % 2 → -1` | `-7 % 2 → 1` (bölenin işareti) |
| Tek mi testi | `x % 2 != 0` (== 1 negatifte yanlış) | C ile aynı | `x % 2 == 1` çalışır |

| Öğe | C | C++ | Python |
|---|---|---|---|
| Tamsayı genişliği | `int32_t, uint64_t …; işaretli taşma tanımsız, işaretsiz mod 2^N` | `C ile aynı (<cstdint>)` | `int keyfi duyarlıklı: taşma yok, daha yavaş` |
| assert | `<assert.h>; -DNDEBUG kapatır` | `<cassert>; -DNDEBUG` | `assert deyimi; python3 -O kapatır` |
| Süre ölçümü | `timespec_get (C11), clock()` | `std::chrono::steady_clock, std::clock` | `time.perf_counter, time.process_time` |
| "Bulunamadı" | `-1 (geçersiz indis)` | `-1` | `None (is None ile sınanır)` |
| Rastgelelik | `rand(): kitaplığa bağlı` | `std::mt19937` | `random (Mersenne Twister)` |

**Tablo 1.17**
*Kütüphane karşılıkları (her dosyanın sonundaki "Kütüphane karşılığı" notu)*

| Öğe | C | C++ | Python |
|---|---|---|---|
| En büyük eleman | `yok (elle)` | `*std::max_element(a.begin(), a.end())` | `max(a)` |
| Doğrusal arama | `yok (POSIX lfind)` | `std::find` | `a.index(x), x in a` |
| EBOB | `yok (elle)` | `std::gcd (<numeric>, C++17)` | `math.gcd(a, b)` |
| Toplam | `yok (elle)` | `std::accumulate (<numeric>)` | `sum(a)` |
| Bölüm ve kalan | `div(a, b) (<stdlib.h>)` | `std::div` | `divmod(a, b) (taban bölmesiyle)` |
| Sıralama (kâhinde) | `qsort` | `std::sort` | `sorted(a)` |

# 9. Mühendis Gözüyle

MÜHENDİS GÖZÜYLE • **Test doğruluğu kanıtlamaz: kâhin, değişmez ve ölçüm birlikte kullanılır**

Üç araç üç farklı soruyu yanıtlar. **Değişmez ve sonlanma kanıtı** "fikir doğru mu?" sorusunu her girdi için yanıtlar ama koddaki yazım hatasını görmez. **Kâhinle karşılaştırmalı test** koddaki hatayı yakalamakta çok güçlüdür ama yalnızca denenen girdiler için konuşur (kâhin bu yüzden farklı ve apaçık bir yoldan hesaplar). **Ölçüm** sürenin kuramla uyumunu ve sabit çarpanları gösterir; Mini-OBS v1.0'ın yükleyicisinin karesel olduğu ancak ölçümle ortaya çıktı (Bölüm 10). Bir fonksiyonu "bitti" saymadan önce:

- Sözleşme yorumda yazılı ve ön koşul assert ile denetleniyor mu?
- Değişmez ve sonlanma niceliği biliniyor mu?
- Sınır değerleri (boş/tek eleman, eşitlik, negatif, uç değerler) test edildi mi?
- Kâhinle rastgele test var mı; tohum kayıtlı mı; test kasıtlı bir hatayla sınandı mı?
- Süre -O2, ısınma + tekrar + ortanca düzeniyle ve sonucu kullanılarak mı ölçüldü?

> MÜHENDİS GÖZÜYLE · **Tekrarlanabilir deney: sabit tohum; derleyici, bayrak ve sürüm kaydı**
>
> Bir ölçüm, başkasının aynı koşullarda tekrarlayabildiği ölçüde değerlidir. Deney kaydı en az şunları içerir: **veri** (üreteç, tohum, n, değer aralığı), **ortam** (işlemci, çekirdek, önbellek, işletim sistemi, yük), **araçlar** (`gcc --version`, `python3 --version`), **bayraklar**, **düzen** (ısınma, tekrar, oturum, ortanca) ve **tarih**. Haftanın Deneyi bunları `deney/OLCUM_ORTAMI.txt`'ye kendiliğinden yazar; tablolar ve grafikler ham CSV'lerden programla üretilir: elle kopyalanan sayı yoktur.

# 10. Mini-OBS 2 Adımı: v2.0 — Veri Üreteci, Ölçüm Düzeni ve Kâhinle Test

Mini-OBS v1.0'ı ölçeklemeye hazırlıyoruz: tekrarlanabilir büyük veri, güvenilir süre ölçümü ve kâhinle test. Kayıt modeline, dosya biçimlerine ve menüye dokunmuyoruz. Mini-OBS 2, Mini-OBS v1.0'ı devralan ve her hafta o haftanın algoritmasıyla büyüyen dönem projesidir (AP-1'de "programı kurmak", AP-2'de "programı ölçeklemek"). v2.0 üç parça ekler: ortak LCG ile $10^3$–$10^6$ kayıtlık, tohumla tekrarlanabilir veri üreten **veri_uret**; 1 ısınma + K tekrar + ortanca ile ölçen **deney** modülü ve **olcer**; istatistik fonksiyonlarını kâhinle sınayan **test_kahin**.

> MİNİ-OBS 2 PROJESİ · **Bu altyapıyla yapılan ilk ölçümün bulgusu**
>
> v1.0'ın yükleyicisi her kaydı eklerken o ana kadarki bütün kayıtları tarar (mükerrer numara kuralı): yükleme **karesel** zamanlıdır. Referans çözümde 1 000 kayıt 1,5 ms'de, 100 000 kayıt 17,1 s'de yüklendi (ortanca; v2.0 ölçüm raporu). Yükleyici bu hafta bilerek değiştirilmez: bulgu H2'de oran testiyle çözümlenecek, yükleme H4'te (v2.3) ve H6'da (v2.5) iyileşir; H10 (v2.9) yükleyiciyi değil, yinelenen tespitini O(n)'e indirir.

### Gereksinimler

- `uretec.c`: `uretec.h`'deki dört fonksiyon. Numaralar `2400000001 … 2400000000 + n` aralığının `rastgele_karistir` ile karıştırılmış hâli; `k` yinelenen için karışık bir sıranın ilk k elemanı hedef, geri kalanlardan `rastgele_aralik` ile seçilen biri kaynak. Üç alt akış: `tohum`, `tohum + 1`, `tohum + 2`. Ad/soyad verilen havuzlardan; yetenek 20–95, vize ± 20, final ± 25, 0–100'e kırpma. `rand()` kullanılmaz.
- `veri_uret.c`: `--n`, `--tohum`, `--yinelenen`, `--cikti` argümanları; geçersiz argümanda kullanım iletisi ve çıkış kodu 2; dosya açılamazsa 1; süre yalnızca stderr'e.
- `dosya.c`: `csv_yaz`'ın içini `csv_basligi_yaz` ve `csv_satiri_yaz` olarak ayırın (çıktı değişmemeli).
- `deney.c`: 1 ısınma + K tekrar; ortanca, ilk, ortalama, en az, en çok (`zaman_sn`, `ortanca` kullanılarak).
- `olcer.c`: en az `yukleme`, `istatistik`, `arama` işlemleri; stdout'a deterministik kontrol değeri, stderr'e ve CSV'ye süre.
- `test_kahin.c`: en az üç fonksiyon için kâhin (ör. `min_max_indis` ↔ sıralama + ilk indis; `dizi_ortalama` ↔ long double toplam; `standart_sapma` ↔ ikili fark özdeşliği); uyuşmazlıkta tohum, n, beklenen/bulunan ve karşı örnek; çıkış kodu 1.
- `Makefile`: `make`, `make test`, `make olcum`, `make asan`, `make temiz`.

### İpuçları

Benzersizlik için "üret ve denetle" yerine **permütasyon** kullanın; denetim istiyorsanız bile önce bunun kaç karşılaştırma tutacağını hesaplayın. Alt akışlar için `rastgele_tohum(tohum + 1)` gibi yeniden

tohumlama yeterlidir; böylece `--yinelenen` ad ve notları değiştirmez. $10^6$ kaydı `Sinif` dizisine toplamayın; satırı üretip hemen `csv_satiri_yaz` ile yazın. Ölçtüğünüz işin sonucunu bir değişkende biriktirip yazdırın; aksi hâlde `-O2` döngüyü silebilir. Kısa işlemleri bir ölçümde $10^3$–$10^4$ kez tekrarlayıp süreyi işlem sayısına bölün. Kâhin, sınanan koddan farklı bir yoldan hesaplamalı; aynı fonksiyonu iki kez çağırmak kâhin değildir. Test boyutlarını deneme sırasından değil **tohumdan** türetin; o zaman "`--tohum t --deneme 1`" hatayı aynen yeniden üretir.

### Kabul ölçütleri

1. `make` ve `make test -std=c17 -Wall -Wextra -pedantic -O2` ile **sıfır uyarı** verir; bütün birim ve kâhin testleri geçer (`test_kahin` varsayılan 300 denemede 0 uyuşmazlık).
2. `./veri_uret --n 3 --tohum 42` çıktısının ilk veri satırı `2400000003;MEHMET;ÜNAL;75;76;40;40;75;63`'tür; `./veri_uret --n 1000 --tohum 42` dosyasının SHA-256 özeti `9135d0d8f8f21c34…` ile başlar (verilen havuzlar ve tanımlı üretim sırasıyla).
3. Aynı argümanlarla iki çalıştırma bayt bayt aynı dosyayı verir (`cmp`); `--n 10^6` dosyası 1 000 001 satırdır ve `sort -t';' -k1,1 -u` ile 1 000 001 benzersiz satır kalır (bu makinede üretim 1,908 s sürdü).
4. `--yinelenen K` dosyası K = 0 dosyasından yalnızca K satırın numara alanında ayrışır; `mini_obs` bu dosyayı "dosya biçimi hatalı (satır s)" ile reddeder ve 1 ile çıkar.
5. Üretilen her dosya `mini_obs` ile yüklenir (ör. 1 000 kayıt: "Yüklendi: 1000 kayıt").
6. `olcer` stdout'u aynı argümanlarla her çalıştırmada aynıdır (yalnızca kontrol değerleri); CSV'de her n için ortanca, en az, en çok ve ilk ölçüm vardır.
7. `--hata-goster` ile kâhin hatalı `min_max_indis`'i yakalar, tohum ve en çok 20 elemanlık bir karşı örnek basar, 1 ile çıkar.
8. Ölçüm raporunda (kendi makinenizde) yükleme için en az altı n değeri ($10^3$ … $10^5$), katlama oranları ve log–log grafik bulunur; $10^6$ için yapılan **kestirim** açıkça "kestirim" diye belirtilir.
9. `make asan` hata ve sızıntı raporlamaz.

### Test senaryoları

**Tablo 1.18**
*Mini-OBS 2 v2.0 test senaryoları*

| # | Girdi | Beklenen çıktı / davranış |
|---|---|---|
| 1 | `./veri_uret --n 5 --tohum 42` | başlık + 5 satır; numaralar 2400000001–2400000005'in bir permütasyonu; çıkış 0 |
| 2 | aynı komut iki kez, `--cikti a.csv` / `b.csv` | `cmp a.csv b.csv` fark bulmaz |
| 3 | `--tohum 43` | `a.csv`'den farklı dosya |
| 4 | `--n 1000 --tohum 42 --yinelenen 3` | `diff` tam 3 satır gösterir; "farklı numara 997" |
| 5 | `./veri_uret --tohum 42` (n yok) | kullanım iletisi, çıkış 2 |
| 6 | `--n 5 --yinelenen 5` | kullanım iletisi, çıkış 2 (K < N olmalı) |
| 7 | `--cikti /olmayan_klasor/x.csv` | "Hata: … açılamadı", çıkış 1 |
| 8 | `printf '6\n10\n' \| ./mini_obs a.csv` | "Yüklendi: 1000 kayıt"; istatistik tablosu |
| 9 | `./mini_obs y3.csv` | "dosya biçimi hatalı (satır s)", program başlamaz, çıkış 1 |
| 10 | `./test_kahin` | "Kâhin testleri geçti: 300 deneme …", çıkış 0 |

| # | Girdi | Beklenen çıktı / davranış |
|---|---|---|
| 11 | `./test_kahin --hata-goster` | "KÂHİN UYUŞMAZLIĞI (tohum t)", küçültülmüş karşı örnek, çıkış 1 |
| 12 | `./olcer yukleme --n 1000,2000,4000 2>/dev/null` | üç satır, `kontrol = n`; her çalıştırmada aynı |
| 13 | `./olcer siralama` | kullanım iletisi, çıkış 2 |

### Başlangıç noktası

**Verilir:** kendi v1.0'ınız (ya da referans v1.0), ortak modüller `ortak_rastgele.h/.c` ve `zamanlayici.h/.c` (değiştirilmez), `uretec.h` (fonksiyon bildirimleri ve üretim sırası), ad/soyad havuzları, `deney.h`, `Makefile` iskeleti. **Yazılır / değiştirilir:** `uretec.c`, `veri_uret.c`, `deney.c`, `olcer.c`, `test_kahin.c` (yeni); `dosya.h/.c` (`csv_yaz`'ın bölünmesi); `test_obs.c` (üreteç testleri); `obs.c`'de yalnızca program başlığı. **Değiştirilmez:** kayıt modeli (`obs.h`'deki yapılar), CSV ve ikili biçim, yükleyici (`csv_oku` — karesel olduğu bilinerek), menü.

# 11. Alıştırmalar ve Atölye

Türler: A) izleme/çıktı tahmini, B) karmaşıklık, C) değişmez yazma, D) hata bulma, E) Parsons, F) kod yazma, G) diller arası okuma. Kodlar `kodlar/alistirmalar/` altındadır; kod yazma iskeletleri derlenir ve testleri başarısız olur. Önce tahmin edin, sonra çalıştırın.

## 11.1 Alıştırmalar

### A · İzleme ve çıktı tahmini

**Alıştırma 1.1** ★☆☆ *Çıktı tahmini*

Çıktıyı çalıştırmadan yazın. `>=` yerine `>` olsaydı `yer` kaç olurdu? m ve yer için değişmezi yazın.

`alistirmalar/H01_A01_cikti_tahmini.c · satır 11–25`

```c
int main(void)
{
    const int a[] = {4, 9, 2, 9, 1};
    int m = a[0];
    size_t yer = 0;
    for (size_t i = 1; i < 5; i++) {
        if (a[i] >= m) {
            m = a[i];
            yer = i;
            printf("i = %zu: m = %d\n", i, m);
        }
    }
    printf("sonuç: m = %d, yer = %zu\n", m, yer);
    return 0;
}
```

**Alıştırma 1.2** ★★☆ *İzleme*

a = [6, 2, 9, 4, 9] için (a) en büyük eleman izleme tablosunu Tablo 1.5 biçiminde; (b) doğrusal aramada x = 4 ve x = 5 için tabloyu ve karşılaştırma sayılarını; (c) ebob(84, 36) için (a, b, a mod b) tablosunu yazın. Her satırın başında değişmezin doğru olduğunu gösterin.

## B · Karmaşıklık

**Alıştırma 1.3** ★☆☆ *Karmaşıklık*

`H01_A03_sayaclar.c` karşılaştırmaları sayar. Çalıştırmadan n = 10 için tahmin edin: `en_buyuk` kaç karşılaştırma yapar; `dogrusal_ara` aranan ilk elemansa ve yoksa kaç? Formülü n cinsinden yazın, en iyi/en kötü durumu Θ ile belirtin; sonra çalıştırıp karşılaştırın.

**Alıştırma 1.4** ★★☆ *Karmaşıklık*

(a) `ikinci_en_buyuk` artan ve azalan sıralı dizide kaç karşılaştırma yapar? En iyi/en kötü durumu n cinsinden yazın. (b) Öklid (144, 89) ve (987, 610) için kaç adım sürer? Fibonacci kuralıyla (Bölüm 4.3) yanıtlayın; adım sayısının neden O(log min(a, b)) olduğunu bir cümleyle açıklayın.

## C · Değişmez yazma

**Alıştırma 1.5** ★★☆ *Değişmez yaz*

`alistirmalar/H01_A05_degismez_yaz.c`'deki iki döngünün (`en_kucuk_indis`: en küçük elemanın ilk indisi; `fibonacci`: x, y ardışık iki Fibonacci sayısı) değişmezini yazın, başlatma–koruma–sonlanma ile gerekçelendirin ve `assert(1)` satırlarını değişmezi denetleyen assert'lerle değiştirin. `en_kucuk_indis`'te `<` yerine `<=` yazılsaydı değişmezin hangi parçası bozulurdu?

**Alıştırma 1.6** ★★★ *Değişmez yaz*

`alistirmalar/H01_A06_basamak_toplami.c`'deki `basamak_toplami(n)` döngüsü (`s += x % 10; x /= 10;`) için değişmezi (ipucu: bt(x), x'in basamak toplamı; s ile bt(x) arasında değişmeyen ilişki) ve sonlanma niceliğini yazın; ikisini assert ile denetleyen bir sürüm yazın.

## D · Hata bulma

**Alıştırma 1.7** ★☆☆ *Hata bul*

Üç test de doğru yanıt veriyor. Hata nedir? Hatayı gösteren en küçük girdiyi yazın. Değişmez kanıtının hangi adımı bozulur?

**alistirmalar/H01_A07_hata_bul.c · satır 13–22**

```
13  int en_buyuk(const int a[], size_t n)
14  {
15      int m = a[0];
16      for (size_t i = 1; i < n - 1; i++) {
17          if (a[i] > m) {
18              m = a[i];
19          }
20      }
21      return m;
22  }
```

**Alıştırma 1.8** ★★☆ *Hata bul*

Sözleşme "x'in İLK konumu, yoksa −1" der; `main`'deki aramalar doğru görünür. Hatayı gösteren bir test tasarlayın ve fonksiyonu düzeltin. Hatalı sürüm hangi durumda en kötü durum olmadığı hâlde n karşılaştırma yapar?

**alistirmalar/H01_A08_son_konum.c · satır 14–23**

```
14  int dogrusal_ara(const int a[], size_t n, int x)
15  {
16      int k = -1;
17      for (size_t i = 0; i < n; i++) {
18          if (a[i] == x) {
19              k = (int)i;
20          }
21      }
22      return k;
23  }
```

## E · Parsons

**Alıştırma 1.9** ★★☆ *Parsons*

Satırları doğru sıraya dizerek `ebob`'u kurun. Bir satır fazladır: hangisi, neden yanlış?

**alistirmalar/H01_A09_parsons_ebob.c · satır 6–15**

```
 6          a = b;
 7      return a;
 8  int ebob(int a, int b)
 9          b = r;
10      }
11  {
12          a = r;
13      while (b != 0) {
14  }
15          int r = a % b;
```

Dosya tamamlanacaktır; bu hâliyle derlenmez.

## F · Kod yazma

**Alıştırma 1.10** ★☆☆ *Kod yaz*

`H01_A10_en_kucuk.c` iskeletindeki `en_kucuk(a, n)`'yi yazın: sözleşmeyi yorumda belirtin, ön koşulu ve değişmezi assert ile denetleyin. `main`'deki beş test (tek eleman, INT_MIN, eşitlik, en küçük sonda) geçmeli. C++ ve Python karşılıklarını da yazın.

**Alıştırma 1.11** ★★☆ *Kod yaz*

`H01_A11_ekok.c`: `ekok(a, b) = a / ebob(a, b) * b`'yi `uint64_t` ile ve kaba kuvvet bir kâhin (a'nın katlarını dene) yazın; 1000 rastgele çiftte (ortak LCG, tohum 2026, a, b ∈ [1, 1000]) karşılaştırın. Neden önce bölmek gerekir? `a * b / ebob(a, b)` ekok($2^{40}$, $2^{40}$) için ne döndürür? Python'da aynı sorun var mı?

**Alıştırma 1.12** ★★★ *Kod yaz*

`H01_A12_min_max.c`: en küçük ve en büyüğü **aynı geçişte**, elemanları ikişer işleyerek bulun (çiftin elemanlarını önce birbiriyle, sonra küçüğünü en küçükle, büyüğünü en büyükle karşılaştırın). Karşılaştırmaları sayıp n = 1000 ve 999 için 2(n − 1) ile kıyaslayın (en çok 3⌊n/2⌋; Cormen vd., 2022). Değişmezi yazın; iki geçişli bir kâhinle 2000 rastgele dizide test edin; C++ ve Python karşılıklarını yazın.

### G · Diller arası okuma

**Alıştırma 1.13** ★★☆ *Diller arası okuma*

Bu C programının ve Python karşılığının (`H01_A13_diller_arasi.py`; C++ sürümü de klasörde) dört satırlık çıktısını çalıştırmadan yazın. Hangi satırlar dile göre değişir, neden? 3. satırda Python'un sonucu C'ninkiyle aynı olsun diye ne yapılmalı?

**alistirmalar/H01_A13_diller_arasi.c · satır 13–22**

```c
int main(void)
{
    printf("1) 7 / 2 = %d\n", 7 / 2);
    printf("2) -17 / 5 = %d, -17 %% 5 = %d\n", -17 / 5, -17 % 5);
    uint32_t x = 4000000000u;
    x = x + 500000000u;
    printf("3) 4000000000 + 500000000 = %" PRIu32 "\n", x);
    printf("4) (int)(-3.7) = %d\n", (int)(-3.7));
    return 0;
}
```

## 11.2 Atölye

Dört atölye etkinliği derste ya da laboratuvarda, eşli ya da küçük gruplarla yapılır; her biri bir alıştırma dosyasıyla başlar ve bir tartışma sorusuyla biter.

### Atölye 1 · "Değişmezi bul" kartları (eşli, ≈ 25 dk)

Altı kısa döngü `alistirmalar/H01_A14_degismez_kartlari.c`'dedir (Tablo 1.19). Eşinizle her kart için: (1) son koşulu yazın; (2) değişmezi bulun ve stratejiyi söyleyin (Bölüm 3.6); (3) `assert(1)` satırını değişmezi **farklı bir yoldan** denetleyen bir assert ile değiştirin; (4) başlatma, koruma ve sonlanmayı eşinize sözlü anlatın; (5) çalıştırın: hiçbir assert patlamamalı. Sonra bilerek bir hata yerleştirin (ör. `c++`'yı silin) ve assert'inizin yakaladığını görün.

**Tablo 1.19**
*Değişmez kartları*

| Kart | Fonksiyon | Ne yapar? | İpucu |
|---|---|---|---|
| 1 | `negatif_say` | a[0..n−1]'deki negatifleri sayar | şimdiye kadar |
| 2 | `sirali_mi` | dizi azalmayan sırada mı? | şimdiye kadar; erken çıkış |
| 3 | `faktoriyel` | n! (n ≤ 20) | k cinsinden; sonlanmada k = n + 1 |
| 4 | `ters_cevir` | diziyi yerinde ters çevirir | iki uç, ortada dokunulmamış bölge |
| 5 | `bir_say` | x'in ikilik gösterimindeki 1'ler | korunan nicelik; x kesin azalır |
| 6 | `var_mi` | x dizide var mı (bayrak)? | bayrak = (x ∈ a[0..i−1]) |

### Atölye 2 · Hata avı: beş fonksiyon, beş hata (≈ 25 dk)

`alistirmalar/H01_A15_hata_avi.c` beş fonksiyon içerir (dosya kasıtlı hata içerir): F1 `toplam`, F2 `dogrusal_ara`, F3 `en_buyuk_indis` (ilk indis), F4 `ortalama`, F5 `ebob`; her birinde **bir** hata ve dosyada doğru bir kâhini (`kahin_*`) vardır. Her fonksiyon için (1) hatayı gösteren **en küçük** girdiyi bulan bir sınır testi, (2) ortak LCG ile (tohum 7, n = 1..8, dar değer aralığı) kâhinle rastgele test yazın; hatayı ve çiğnediği sözleşme maddesini bir cümleyle yazın. F1 için örnek sınır testi:

C **alistirmalar/H01_A15_hata_avi.c · satır 125–129**

```c
    /* Örnek sınır testi (F1): tek elemanlı dizi */
    const int t[] = {5};
    long long beklenen = kahin_toplam(t, 1), bulunan = toplam(t, 1);
    printf("F1 sınır testi [5]: beklenen %lld, bulunan %lld -> %s\n",
           beklenen, bulunan, beklenen == bulunan ? "geçti" : "YAKALANDI");
```

ÇIKTI

```
$ ./H01_A15_hata_avi
F1 sınır testi [5]: beklenen 5, bulunan 0 -> YAKALANDI
```

Tartışma: hangi hataları sınır testi, hangilerini rastgele test önce buldu? Değer aralığı genişleyince (ör. [−1000, 1000]) hangi hatanın yakalanma sıklığı düşer, neden?

### Atölye 3 · Ölçüm deneyi: üç dil, iki optimizasyon düzeyi (≈ 20 dk)

Örnek 13'ü dört yapılandırmada derleyip (C -O0, C -O2, C++ -O2, Python; komutlar Bölüm 3.10'daki kalıpla, C'de `ortak_rastgele.c zamanlayici.c` eklenerek) `--nmax 1000000 --csv` ile çalıştırın ve `H01_A16_olcum_birlestir.py` ile birleştirin. Önce tahmin edin: n on katına çıkınca süre kaç katına çıkar? Python, C -O2'den kaç kat yavaştır?

ÇIKTI · BU MAKİNEDE, TEK BİR DENEME (YALNIZCA PYTHON SATIRLARI VE EĞİMLER)

```
$ python3 H01_A16_olcum_birlestir.py c_O0.csv c_O2.csv cpp_O2.csv py.csv
yapılandırma            n  ortanca (ms)  T(10n)/T(n)
C O0           log-log eğim = 1.00
C O1+          log-log eğim = 1.00
C++ O1+        log-log eğim = 0.96
Python CPython       1000        0.0335            –
Python CPython      10000        0.3516        10.49
Python CPython     100000        6.3339        18.01
Python CPython    1000000       38.4431         6.07
Python CPython log-log eğim = 1.04
```

Betik her yapılandırma için T(10n)/T(n) oranını ve log–log eğimini hesaplar. Bu deneme Bölüm 6'nın dersini kendiliğinden verdi: Python'da n = 100 000'deki tek bir gecikmiş ölçüm iki oranı birden bozdu (18,01 ve 6,07; kuram 10). Tutarsız bir satır görürseniz o n için ölçümü (oturumu) tekrarlayın ve iki sonucu da raporlayın; $n = 10^3$'te -O2 ile derlenmiş C ve C++ ölçümleri yarım mikrosaniye mertebesindedir ve gürültüye en duyarlı olanlardır. Tabloyu web sayfasındaki ölçüm laboratuvarına yükleyip eğimi Bölüm 6 ile karşılaştırın.

### Atölye 4 · Üç dilli ortamın kurulumu ve esdeger_test.py (≈ 15 dk)

1. **Derleyiciler:** Windows'ta MSYS2 (UCRT64) ile `mingw-w64-ucrt-x86_64-toolchain` paketini kurun ve `ucrt64\bin`'i PATH'e ekleyin; Linux/WSL'de `sudo apt install build-`

`essential python3`; macOS'ta `xcode-select --install` (`gcc`/`g++` Clang'ı çağırır). Python'u python.org'dan kurun (Windows'ta "Add python.exe to PATH").

2. **Doğrulama:** `gcc --version`, `g++ --version`, `python3 --version` (Windows'ta `py --version`) çıktılarını not edin: deney kaydınızın ilk satırlarıdır.
3. **Kodlar:** `AP2_H01_kodlar.zip`'i açın ve `kodlar/` klasörüne geçin.
4. **Eşdeğerlik:** `python3 esdeger_test.py .` her örneği üç dilde derler, çalıştırır ve `ciktilar/` ile karşılaştırır; sonuç "Toplam 13 örnek · geçen 13 · hatalı 0" olmalıdır.
5. **Sorun giderme:** Türkçe karakterler bozuksa `chcp 65001` ve `set PYTHONUTF8=1`; derleyici bulunamazsa PATH'i, bir örnek geçmezse derleyici sürümünü ve satır sonlarını (CRLF/LF) denetleyin.
6. **Değiştir:** `c/H01_06_bolme_kalan.c`'de `int q = a / b;` satırını `int q = (a - a % b) / b;` yapıp betiği yeniden çalıştırın: çıktı değişir mi, neden? (Sonra geri alın.)

## 12. Kendini Değerlendirme

- ☐ Bir hesaplama problemini girdi kümesi, çıktı koşulu ve örnekle tanımlayabiliyorum (3.1).
- ☐ Kısmi doğruluk, sonlanma ve tam doğruluk farkını bir örnekle açıklayabiliyorum (3.2).
- ☐ Ön/son koşul yazıp assert ile denetleyebiliyorum; assert'in ne zaman kapandığını biliyorum (3.3, Örnek 2).
- ☐ Bir döngünün değişmezini son koşuldan türetip başlatma–koruma–sonlanma ile gerekçelendirebiliyorum (3.5–3.6).
- ☐ Öklid'in neden sonlandığını azalan nicelikle gösterebiliyorum (4.3).
- ☐ Üç amiral algoritmanın en iyi/en kötü durumunu ve kimlik kartını yazabiliyorum (Bölüm 4).
- ☐ Sınır değer, rastgele ve kâhinle karşılaştırmalı test yazıp testimi kasıtlı bir hatayla sınayabiliyorum (3.8, Örnek 9–10).
- ☐ Isınma, tekrar ve ortancayla süre ölçebiliyor; -O2'nin ölçümü nasıl bozabileceğini açıklayabiliyorum (3.9, Örnek 11–12).
- ☐ −7 / 2, −7 % 2, 13! ve `x % 2 == 1` için üç dilin davranışını açıklayabiliyorum (Örnek 6–7, Bölüm 8).
- ☐ Bir deney kaydına neleri yazmam gerektiğini biliyorum (Bölüm 9).

## 13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · **Bu haftanın konusu nerede derinleşiyor?**

- **Döngü değişmezleri** H4 (ikili arama: aranan a[lo..hi] içindeyse), H5 (araya ekleme: a[0..i−1] sıralı), H6 (hızlı sıralamada bölümleme) ve H12'nin (Dijkstra: kesinleşmiş köşelerin uzaklıkları doğru) doğruluk gerekçelerinin ortak dilidir.
- **Kâhinle karşılaştırmalı test** dönem boyunca her yeni algoritmanın kabul ölçütüdür.
- **Hoare mantığı ve araç destekli biçimsel doğrulama** (Timsort öyküsündeki KeY, C için Frama-C) ileri yazılım mühendisliği derslerinin konusudur: değişmezler orada makinenin denetlediği iddialardır.

MERAK KUTUSU · **Sonlandığı kanıtlanamamış bir döngü: Collatz**

n çiftse n/2, tekse 3n + 1 yapan ve n = 1'de duran döngü denenen her başlangıç değerinde 1'e inmiştir; ama her n için sonlandığı **kanıtlanmış değildir** (Collatz sanısı): n kimi zaman büyür, kimi zaman küçülür ve kesin azalan bir nicelik bilinmez.

**kavram/H01_K03_collatz.c · satır 15–27**

```
15  long collatz_adim(unsigned long long n, unsigned long long *tepe)
16  {
17      long adim = 0;
18      *tepe = n;
19      while (n != 1) {
20          n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
21          if (n > *tepe) {
22              *tepe = n;
23          }
24          adim++;
25      }
26      return adim;
27  }
```

Programın çıktısına göre 27, 1'e 111 adımda iner ve yol boyunca 9 232'e kadar çıkar. İkinci merak konusu: **genişletilmiş Öklid algoritması** ebob(a, b) = a·x + b·y'yi sağlayan x ve y'yi de bulur; modüler ters bulmanın, dolayısıyla açık anahtarlı şifrelemenin temel adımıdır.

# 14. Özet, Kavram Haritası ve Terim Sözlüğü

Haftanın özü aşağıdaki altı maddededir; kavramlar arasındaki ilişkiler Şekil 1.11'deki haritada, Türkçe–İngilizce terimler Tablo 1.20'de toplanmıştır.

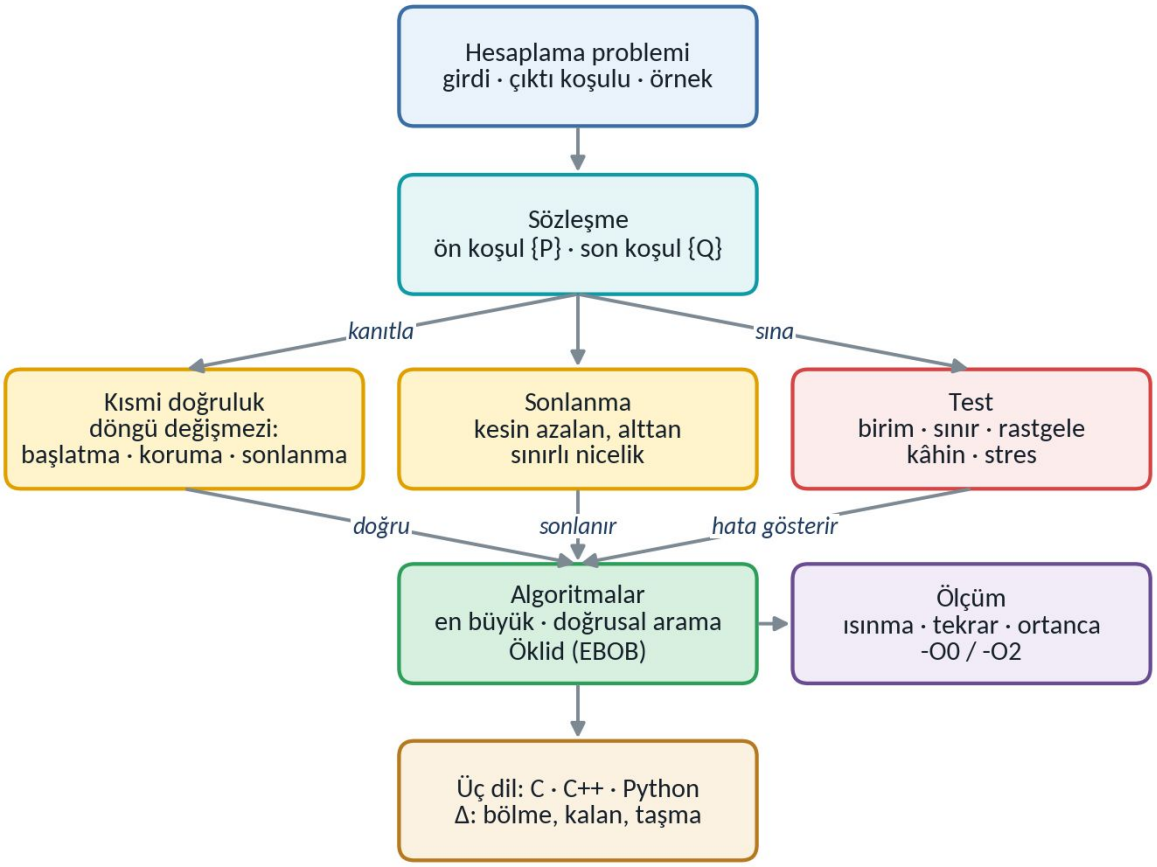
- **Algoritma** her örnekte durup doğru çıktıyı veriyorsa doğrudur: doğruluk = **kısmi doğruluk + sonlanma**. **Sözleşme**: ön koşul çağıranın, son koşul fonksiyonun sorumluluğu; **Hoare üçlüsü** {P} S {Q}.
- **Döngü değişmezi** her yinelemenin başında doğrudur; **başlatma, koruma, sonlanma** ile kanıtlanır ve çoğunlukla son koşuldan türetilir. **Sonlanma** kesin azalan, alttan sınırlı bir nicelikle gösterilir.
- En büyük eleman Θ(n); doğrusal arama en iyi O(1), en kötü Θ(n); Öklid O(log min(a, b)), en kötü durumu ardışık Fibonacci sayıları.
- **Test** hataların varlığını gösterir, yokluğunu kanıtlamaz: sınır değer, rastgele (sabit tohum) ve **kâhinle karşılaştırmalı** test; testin kendisi de sınanır.
- **Ölçüm**: monotonik saat, ısınma, tekrar, **ortanca**, -O2, sonucu kullan; sonuçlar CSV'ye, ortam deney kaydına.
- **Üç dil**: C/C++ sıfıra doğru keser, Python // aşağı yuvarlar; işaretli taşma C/C++'ta tanımsız davranış, Python int'i keyfi duyarlıklı; sabit çarpanlar bir ila iki mertebe farklı.

**Tablo 1.20**
*Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)*

| Türkçe | İngilizce | Türkçe | İngilizce |
|---|---|---|---|
| hesaplama problemi | computational problem | stres testi | stress test |
| örnek (problem örneği) | instance | karşı örnek | counterexample |
| doğruluk | correctness | en iyi / ortalama / en kötü durum | best / average / worst case |

| Türkçe | İngilizce | Türkçe | İngilizce |
|---|---|---|---|
| kısmi / tam doğruluk | partial / total correctness | duvar saati | wall-clock time |
| sonlanma | termination | monotonik saat | monotonic clock |
| ön koşul / son koşul | precondition / postcondition | işlemci zamanı | CPU time |
| sözleşme ile tasarım | design by contract | ısınma | warm-up |
| Hoare üçlüsü | Hoare triple | ortanca | median |
| döngü değişmezi | loop invariant | tanımsız davranış | undefined behavior |
| başlatma / koruma | initialization / maintenance | taşma | overflow |
| sonlanma ölçütü (azalan nicelik) | variant (ranking function) | taban bölmesi | floor division |
| iddia, assert | assertion | sıfıra doğru kesme | truncation toward zero |
| birim testi | unit test | doğrusal eşlik üreteci | linear congruential generator (LCG) |
| sınır değer testi | boundary value testing | tohum | seed |
| kâhin | test oracle | bayt kodu | bytecode |
| karşılaştırmalı test | differential testing | kaba kuvvet | brute force |

**Şekil 1.11**
*Haftanın kavram haritası*



# 15. Kaynaklar

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Dahl, O.-J., Dijkstra, E. W., & Hoare, C. A. R. (1972). *Structured programming*. Academic Press.

de Gouw, S., Rot, J., de Boer, F. S., Bubel, R., & Hähnle, R. (2015). OpenJDK's java.utils.Collection.sort() is broken: The good, the bad and the worst case. In D. Kroening & C. S. Păsăreanu (Eds.), *Computer aided verification: 27th International Conference, CAV 2015* (pp. 273–289). Springer. https://doi.org/10.1007/978-3-319-21690-4_16

Hoare, C. A. R. (1969). An axiomatic basis for computer programming. *Communications of the ACM*, *12*(10), 576–580. https://doi.org/10.1145/363235.363259

ISO/IEC. (2017). *Programming languages — C++* (ISO/IEC Standard No. 14882:2017). International Organization for Standardization.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Python Software Foundation. (t.y.). *The Python standard library*. Erişim tarihi 12 Ekim 2026, https://docs.python.org/3/library/

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Tshukudu, E., & Cutts, Q. (2020). Understanding conceptual transfer for students learning new programming languages. In *Proceedings of the 2020 ACM Conference on International Computing Education Research* (pp. 227–237). ACM. https://doi.org/10.1145/3372782.3406270

van Rossum, G., Warsaw, B., & Coghlan, N. (2001). *PEP 8 – Style guide for Python code*. Python Software Foundation. https://peps.python.org/pep-0008/

# Ek A. Dil Karşılıkları

Bu ek, örnek merdiveninin on üç basamağının C++ ve Python karşılıklarını gösterir. Her örnekte, C sürümünden **farklılaşan satırları** (ornekler.json, web sayfasındaki "farkları göster") en çok içeren kesit verilmiştir; hardal satırlar dil farkıdır, satır numaraları dosyalarla aynıdır. Kesit dışında kalan fark satırları küçük bir tabloda listelenir. Dosyaların tamamı `kodlar/cpp/` ve `kodlar/python/` klasörlerinde ve web sayfasının üç dilli kod sekmelerindedir; derleme komutları Tablo 1.4'te, ortak modül kullanan örnekler (8–13) her dosyanın başlık yorumundadır. Δ örneklerinin (6 ve 7) üç dildeki çıktıları da buradadır.

## A.1 Örnek 1 · Dizinin en büyük elemanı — ön/son koşul yorumlu fonksiyon (★, =)

C++ **H01_01_en_buyuk.cpp · satır 19–31**

```cpp
// denetimi yapmaz, davranış tanımsızdır (a.at(0) olsaydı
// std::out_of_range istisnası atılırdı).
int en_buyuk(const std::vector<int>& a)
{
    const std::size_t n = a.size();
    int m = a[0];
    for (std::size_t i = 1; i < n; i++) {
        if (a[i] > m) {
            m = a[i];
        }
    }
    return m;
}
```

Python **H01_01_en_buyuk.py · satır 14–28**

```python
# Ön koşul ihlali (boş liste) çağıranın hatasıdır: a[0] okunurken
# IndexError istisnası oluşur; fonksiyon bunu ayrıca denetlemez.
def en_buyuk(a):
    """a'nın en büyük elemanı."""
    m = a[0]
    for i in range(1, len(a)):
        if a[i] > m:
            m = a[i]
    return m


def sonuc_yaz(ad, a):
    """ad = [x, y, ...] -> en büyük: m biçiminde yazdırır."""
    metin = ", ".join(str(x) for x in a)
    print(f"{ad} = [{metin}] -> en büyük: {en_buyuk(a)}")
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de dizi ve boyutu ayrı parametredir (`const int a[], size_t n`); C++'ta `const std::vector<int>&` boyutu `a.size()` ile taşır, Python listesi `len(a)` ile. Ön koşul ihlalinde (boş dizi) C ve C++'ta `a[0]` tanımsız davranıştır; Python `IndexError` istisnası verir.

## A.2 Örnek 2 · Aynı fonksiyonda döngü değişmezinin assert ile denetlenmesi (★, =)

C++ **H01_02_en_buyuk_assert.cpp · satır 10–28**

```cpp
#include <cassert>
#include <cstddef>
#include <iostream>
#include <string>
#include <vector>

// Yalnızca gösterim için: degismez() kaç kez çağrıldı? -DNDEBUG ile
// derlenince assert(...) içindeki ifade HİÇ değerlendirilmez ve bu
// sayaç 0 kalır. Programın asıl işi assert içine konmamalıdır.
static long denetim_sayisi = 0;

// degismez: m, a[0..i-1] bölümünün en büyüğü mü? (0(i) denetim)
// m bu bölümde geçmeli ve bölümdeki hiçbir eleman m'den büyük
// olmamalıdır.
bool degismez(const std::vector<int>& a, std::size_t i, int m)
{
    bool gecti_mi = false;
    denetim_sayisi++;
    for (std::size_t k = 0; k < i; k++) {
```

Python **H01_02_en_buyuk_assert.py · satır 34–46**

```python
    """Ön koşul: len(a) >= 1. Son koşul: m = max(a[0..n-1])."""
    n = len(a)
    assert n >= 1                    # ön koşul
    m = a[0]
    i = 1
    assert degismez(a, i, m)         # başlatma
    while i < n:
        if a[i] > m:
            m = a[i]
        i += 1
        assert degismez(a, i, m)     # koruma
    assert i == n                    # sonlanma: i == n
    return m  # değişmez + (i == n)  =>  m = max(a[0..n-1])
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de `assert <assert.h>`, C++'ta `<cassert>` makrosudur ve `-DNDEBUG` ile derlenince ifade hiç değerlendirilmez; Python'da `assert` bir deyimdir ve `python3 -O` ile kapanır. Üç dilde de kapatılınca sayaç 0 olur: assert içine programın asıl işi konmaz.

| Dil | Satır | Kesit dışındaki farklı satır |
|---|---|---|
| Python | 22 | `global denetim_sayisi` |

### A.3 Örnek 3 · 1..n toplamı: döngü ve formül; değişmezle doğruluk (★, =)

C++ H01_03_toplam_1_n.cpp · satır 31–34

```cpp
long long toplam_formul(int n)
{
    return static_cast<long long>(n) * (n + 1) / 2;
}
```

Python H01_03_toplam_1_n.py · satır 17–29

```python
    """1 + 2 + ... + n (döngüyle)."""
    s = 0
    for k in range(1, n + 1):
        s += k
    return s


# toplam_formul: Gauss formülü n(n+1)/2. Python int'i keyfi
# duyarlıklıdır: taşma yoktur. // tamsayı bölmesidir (/ ondalık sayı,
# float, verirdi).
def toplam_formul(n):
    """Gauss formülü n(n+1)/2."""
    return n * (n + 1) // 2
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C ve C++'ta `n * (n + 1)` çarpımı `long long`'a dönüştürülmeden yapılırsa n = 65536'da 32 bitlik `int` taşar (tanımsız davranış); Python `int`'i taşmaz. Python'da `/` ondalık sayı verir, tamsayı bölmesi `//` ile yapılır (negatif olmayan sayılarda C'deki `/` ile aynı sonuç).

| Dil | Satır | Kesit dışındaki farklı satır |
|---|---|---|
| C++ | 64 | `for (int n : n_deger) {` |

### A.4 Örnek 4 · Doğrusal arama: değişmez ve "bulunamadı" sözleşmesi (−1 / None) (★★, =)

C++ H01_04_dogrusal_arama.cpp · satır 21–34

```cpp
// Değişmez : döngü başında x, a[0..i-1] içinde yok.
// Sonlanma : i her adımda 1 artar; en çok n adımda i == n olur.
int dogrusal_ara(const std::vector<int>& a,
                 int x)
{
    const std::size_t n = a.size();
    for (std::size_t i = 0; i < n; i++) {
        karsilastirma++;
        if (a[i] == x) {    // 1. çıkış
            return static_cast<int>(i);
        }
    }
    return -1;              // 2. çıkış
}
```

Python **H01_04_dogrusal_arama.py · satır 20–39**

```python
def dogrusal_ara(a, x):
    """x'in ilk konumu; yoksa None."""
    global karsilastirma
    for i in range(len(a)):
        karsilastirma += 1
        if a[i] == x:    # 1. çıkış
            return i
    return None          # 2. çıkış


def ara_yaz(a, x):
    """Aramayı yapar; sonucu ve karşılaştırma sayısını yazar."""
    global karsilastirma
    karsilastirma = 0
    k = dogrusal_ara(a, x)
    if k is None:
        print(f"ara({x}) -> bulunamadı", end="")
    else:
        print(f"ara({x}) -> {k}", end="")
    print(f", karşılaştırma: {karsilastirma}")
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> "Bulunamadı" C ve C++'ta geçersiz bir indisle (`-1`), Python'da `None` ile bildirilir ve `is None` ile sınanır. Python'da modül düzeyindeki sayaca fonksiyon içinden yazmak için `global` gerekir.

| Dil | Satır | Kesit dışındaki farklı satır |
|---|---|---|
| C++ | 59 | `ara_yaz({}, 7);  // boş vektör: döngüye hiç girilmez` |

## A.5 Örnek 5 · Öklid EBOB: azalan nicelikle sonlanma kanıtı (★★, =)

C++ **H01_05_oklid_ebob.cpp · satır 63–72**

```cpp
    ebob_izle(252, 105);
    std::cout << "\n        a         b     ebob  adım\n";
    for (const auto& [a, b] : ciftler) {
        int d = ebob(a, b);
        std::cout << std::setw(9) << a << " " << std::setw(9) << b << " "
                  << std::setw(8) << d << " " << std::setw(5) << adim_sayisi
                  << "\n";
    }
    return 0;
}
```

Python **H01_05_oklid_ebob.py · satır 21–28**

```python
def ebob(a, b):
    """Öklid algoritmasıyla ebob(a, b)."""
    global adim_sayisi
    adim_sayisi = 0
    while b != 0:
        a, b = b, a % b
        adim_sayisi += 1
    return a
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Python'un çoklu ataması `a, b = b, a % b` geçici değişkene gerek bırakmaz; sağ taraf önce bütünüyle hesaplanır. Negatif olmayan sayılarda `%` üç dilde aynı sonucu verir (negatiflerde fark: Örnek 6).

| Dil | Satır | Kesit dışındaki farklı satır |
|---|---|---|
| Python | 50 | `for a, b in ciftler:` |

## A.6 Örnek 6 · -7 / 2 ve -7 % 2: sıfıra doğru kesme ve taban bölmesi (★★, Δ)

C++ **H01_06_bolme_kalan.cpp · satır 15–32**

```cpp
int main()
{
    const std::vector<int> a_deger = {7, -7, 7, -7};
    const std::vector<int> b_deger = {2, 2, -2, -2};

    std::cout << "C++: / sıfıra doğru keser, % bölünenin işaretini alır\n";
    std::cout << "   a    b   a / b   a % b   (a / b) * b + a % b\n";
    for (std::size_t i = 0; i < a_deger.size(); i++) {
        int a = a_deger[i];
        int b = b_deger[i];
        int q = a / b;      // bölüm: sıfıra doğru kesme (C ile aynı)
        int r = a % b;      // kalan: işareti a'nınki    (C ile aynı)
        std::cout << std::setw(4) << a << " " << std::setw(4) << b << " "
                  << std::setw(7) << q << " " << std::setw(7) << r << " "
                  << std::setw(13) << q * b + r << "\n";
    }
    // Yaygın tuzak: x % 2 == 1 negatif tek sayılarda yanlıştır.
    std::cout << "-7 tek mi? (-7 % 2 == 1): "
```

Python **H01_06_bolme_kalan.py · satır 20–32**

```python
        a = a_deger[i]
        b = b_deger[i]
        q = a // b          # bölüm: taban bölmesi (aşağı yuvarlama)
        r = a % b           # kalan: işareti b'ninki
        print(f"{a:4d} {b:4d} {q:7d} {r:7d} {q * b + r:13d}")
    # Python'da x % 2 == 1 negatif tek sayılarda da doğru çalışır.
    print(f"-7 tek mi? (-7 % 2 == 1): {'evet' if -7 % 2 == 1 else 'hayır'}")
    print(f"-7 tek mi? (-7 % 2 != 0): {'evet' if -7 % 2 != 0 else 'hayır'}")
    # C'deki sıfıra doğru kesmenin taklidi (isteğe bağlı):
    # int(a / b) ondalık bölme yapar; çok büyük tamsayılarda (2**53
    # üstü) duyarlık kaybı olabilir. math.fmod C gibi kalan verir.
    print(f"C gibi: int(-7 / 2) = {int(-7 / 2)}, "
          f"math.fmod(-7, 2) = {math.fmod(-7, 2)}")
```

**DİL PENCERESİ · Farklılaşan satırlar**

C (C99'dan beri) ve C++ (C++11'den beri) tamsayı bölmesini sıfıra doğru keser: `-7 / 2` → -3, `-7 % 2` → -1. Python `//` aşağı yuvarlar: `-7 // 2` → -4, `-7 % 2` → 1 (kalan bölenin işaretini alır). Üç dilde de `(a / b) * b + a % b == a` özdeşliği korunur; bu yüzden `x % 2 == 1` testi C/C++'ta negatif tek sayıyı kaçırır.

**ÇIKTI · C++**

```
$ ./H01_06_bolme_kalan
C++: / sıfıra doğru keser, % bölünenin işaretini alır
   a    b   a / b   a % b   (a / b) * b + a % b
   7    2       3       1             7
  -7    2      -3      -1            -7
   7   -2      -3       1             7
  -7   -2       3      -1            -7
-7 tek mi? (-7 % 2 == 1): hayır
-7 tek mi? (-7 % 2 != 0): evet
```

**ÇIKTI · PYTHON**

```
$ python3 H01_06_bolme_kalan.py
Python: // aşağı yuvarlar, % bölenin işaretini alır
   a    b  a // b   a % b  (a // b) * b + a % b
   7    2       3       1             7
  -7    2      -4       1            -7
   7   -2      -4      -1             7
  -7   -2       3      -1            -7
-7 tek mi? (-7 % 2 == 1): evet
-7 tek mi? (-7 % 2 != 0): evet
C gibi: int(-7 / 2) = -3, math.fmod(-7, 2) = -1.0
```

### A.7 Örnek 7 · 13! hesabı: uint32_t sarması ve keyfi duyarlıklı tamsayı (★★, Δ)

C++ H01_07_faktoriyel_tasma.cpp · satır 35–46

```cpp
    }
    std::cout << "13! mod 2^32 = " << f64 << " - "
              << (std::uint64_t{1} << 32) << " = " << f32 << "\n";
    for (std::uint64_t n = 14; n <= 21; n++) {
        f64 *= n;            // 2^64 modunda: 21!'de sarar
        if (n == 20) {
            std::cout << "20! (uint64_t) = " << f64 << "\n";
        }
    }
    std::cout << "21! (uint64_t) = " << f64 << "  <- uint64_t de sardı\n";
    return 0;
}
```

Python H01_07_faktoriyel_tasma.py · satır 14–32

```python
def main():
    f = 1      # Python int: sınırsız büyüklük (bellek yettikçe)
    f32 = 1    # & MASKE32 ile 2^32 modunda tutulan taklit

    print("Python: int keyfi duyarlıklıdır (taşma yok)")
    print(" n           int  & 0xFFFFFFFF")
    for n in range(1, 14):
        f *= n
        f32 = (f32 * n) & MASKE32      # C'deki uint32_t sarması
        isaret = "" if f32 == f else "  <- 32 bit taklidi sardı"
        print(f"{n:2d} {f:13d} {f32:13d}{isaret}")
    print(f"13! mod 2^32 = {f} - {1 << 32} = {f32}")
    for n in range(14, 22):
        f *= n
        if n == 20:
            print(f"20! = {f}")
    print(f"21! = {f}  (sarma yok: keyfi duyarlık)")
    print(f"21! & (2**64 - 1) = {f & MASKE64}  (uint64_t taklidi)")
    print(f"21! için gereken bit sayısı: {f.bit_length()}")
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C ve C++'ta işaretsiz tamsayılar 2^N modunda sarar (tanımlı): 13! `uint32_t`'de 1932053504 olur, `uint64_t` 20!'e kadar doğrudur. İşaretli taşma ise tanımsız davranıştır ve çalıştırılmaz. Python `int`'i keyfi duyarlıklıdır: 21! = 51090942171709440000 tam hesaplanır; sarma ancak `& 0xFFFFFFFF` maskesiyle taklit edilir.

ÇIKTI · C++

```
$ ./H01_07_faktoriyel_tasma
C++: işaretsiz sabit genişlikli tamsayılar (2^N modunda sarar)
 n      uint32_t              uint64_t
 1             1                     1
 2             2                     2
 3             6                     6
 4            24                    24
 5           120                   120
 6           720                   720
 7          5040                  5040
 8         40320                 40320
 9        362880                362880
10       3628800               3628800
11      39916800              39916800
12     479001600             479001600
13    1932053504            6227020800  <- uint32_t sardı
13! mod 2^32 = 6227020800 - 4294967296 = 1932053504
20! (uint64_t) = 2432902008176640000
21! (uint64_t) = 14197454024290336768  <- uint64_t de sardı
```

ÇIKTI · PYTHON

```
$ python3 H01_07_faktoriyel_tasma.py
Python: int keyfi duyarlıklıdır (taşma yok)
 n           int  & 0xFFFFFFFF
 1             1             1
 2             2             2
 3             6             6
 4            24            24
 5           120           120
 6           720           720
 7          5040          5040
 8         40320         40320
 9        362880        362880
10       3628800       3628800
11      39916800      39916800
12     479001600     479001600
13    6227020800    1932053504  <- 32 bit taklidi sardı
13! mod 2^32 = 6227020800 - 4294967296 = 1932053504
20! = 2432902008176640000
21! = 51090942171709440000  (sarma yok: keyfi duyarlık)
21! & (2**64 - 1) = 14197454024290336768  (uint64_t taklidi)
21! için gereken bit sayısı: 66
```

## A.8 Örnek 8 · Ortak LCG modülü ile üç dilde aynı “rastgele” dizi (★★, =)

C++ **H01_08_ortak_lcg.cpp · satır 56–76**

```
56      std::cout << "\n";
57
58      std::vector<int> a = rastgele_dizi(8, -50, 50);
59      std::cout << "rastgele_dizi(8, -50, 50)   = ";
60      dizi_yaz(a);
61
```

```cpp
    std::vector<int> p(10);
    for (int i = 0; i < 10; i++) {
        p[i] = i + 1;
    }
    rastgele_karistir(p);
    std::cout << "rastgele_karistir(1..10)    = ";
    dizi_yaz(p);

    // 3) Aynı tohum → aynı dizi
    rastgele_tohum(42);
    std::uint32_t ilk = rastgele_sayi();
    std::cout << "Yeniden tohum 42, ilk sayı  = " << ilk
              << " (aynı dizi baştan)\n";
    return 0;
}
```

Python **H01_08_ortak_lcg.py · satır 10–26**

```python
from ortak_rastgele import (rastgele_aralik, rastgele_dizi,
                            rastgele_karistir, rastgele_kesir,
                            rastgele_sayi, rastgele_tohum)


def dizi_yaz(a):
    """a listesini "[x, y, ...]" biçiminde yazdırır."""
    print("[" + ", ".join(str(x) for x in a) + "]")


def main():
    # 1) Formülü elle uygula: durum = durum * A + C (mod 2^64)
    durum = 42
    durum = (durum * 6364136223846793005
             + 1442695040888963407) & (2**64 - 1)  # sarmayı taklit et
    print(f"Elle   : durum = {durum}")
    print(f"         çıktı = durum >> 33 = {durum >> 33}")
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de `uint64_t` çarpımı 2^64 modunda kendiliğinden sarar; Python'da aynı LCG adımı `& (2**64 - 1)` maskesi olmadan sonsuz büyürdü. C sürümü `rastgele_dizi(a, n, alt, ust)` ile verilen diziyi doldurur, C++ ve Python sürümleri yeni vektör/liste döndürür; sayılar üç dilde bayt bayt aynıdır.

| Dil | Satır | Kesit dışındaki farklı satır |
|---|---|---|
| C++ | 16 | `#include "ortak_rastgele.hpp"` |
| Python | 46 | `a = rastgele_dizi(8, -50, 50)` |

### A.9 Örnek 9 · Hatalı en büyük (0 ile başlatılmış) fonksiyonun sınır ve rastgele testle yakalanması (★★, =)

C++ **H01_09_hatali_en_buyuk.cpp · satır 75–79**

```cpp
    for (int t = 1; t <= deneme; t++) {
        auto n = static_cast<std::size_t>(rastgele_aralik(1, MAKS_N));
        std::vector<int> a = rastgele_dizi(n, alt, ust);
        int beklenen = en_buyuk(a);
        int bulunan = en_buyuk_hatali(a);
```

Python **H01_09_hatali_en_buyuk.py · satır 57–61**

```python
    for t in range(1, deneme + 1):
        n = rastgele_aralik(1, MAKS_N)
        a = rastgele_dizi(n, alt, ust)
        beklenen = en_buyuk(a)
        bulunan = en_buyuk_hatali(a)
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Hata dilden bağımsızdır; üç dilde aynı girdilerde aynı biçimde ortaya çıkar. Ortak LCG sayesinde rastgele testin ilk karşı örneği (deneme 17: `[-77]`) üç dilde de aynıdır. C++ ve Python'da test dizileri `{...}` / `[...]` ile doğrudan argüman olarak yazılabilir.

| Dil | Satır | Kesit dışındaki farklı satır |
|---|---|---|
| C++ | 102 | `test_et({-8, -3, -12}, "tamamı negatif");` |
| Python | 78 | `test_et([-8, -3, -12], "tamamı negatif")` |

### A.10 Örnek 10 · İkinci en büyük eleman: hızlı fonksiyonun sıralama tabanlı kâhinle karşılaştırılması (★★★, =)

C++ **H01_10_ikinci_en_buyuk.cpp · satır 64–70**

```cpp
// kahin: yavaş ama apaçık doğru — kopyayı sırala, sondan ikinciyi al.
// O(n log n); std::sort yalnızca kâhinde serbesttir.
int kahin(std::vector<int> kopya)   // değerle: kopya alınır
{
    std::sort(kopya.begin(), kopya.end());
    return kopya[kopya.size() - 2];
}
```

Python **H01_10_ikinci_en_buyuk.py · satır 57–69**

```
57      0(n log n); sorted yalnızca kâhinde serbesttir.
58      """
59      return sorted(a)[len(a) - 2]
60
61
62  def dizi_metni(a):
63      """a listesini "[x, y, ...]" biçiminde metne çevirir."""
64      return "[" + ", ".join(str(x) for x in a) + "]"
65
66
67  def karsilastirmali_test(f, ad, deneme, alt, ust):
68      """f'yi kâhine karşı deneme kez sınar (n = 2..MAKS_N, değerler
69      [alt, ust], tohum 2027). İlk uyuşmazlığı yazdırır; uyuşmazlık
```

**DİL PENCERESİ · Farklılaşan satırlar**

Kâhin C'de `qsort` + karşılaştırıcı fonksiyon ve `malloc`'lu kopya; C++'ta değerle alınan vektörün `std::sort`'u; Python'da `sorted(a)` (yalnızca kâhinde serbest). Sınanan fonksiyon C/C++'ta fonksiyon işaretçisiyle, Python'da doğrudan nesne olarak geçirilir; ilk iki elemanın yer değiştirmesi C'de iki atamayla, C++'ta `std::swap`, Python'da `b1, b2 = b2, b1`.

| Dil | Satır | Kesit dışındaki farklı satır |
|---|---|---|
| C++ | 33 | `std::swap(b1, b2);` |
| C++ | 85 | `int karsilastirmali_test(int (*f)(const std::vector<int>&),` |
| Python | 27 | `b1, b2 = b2, b1` |

## A.11 Örnek 11 · Süre ölçümü: clock/timespec_get, std::chrono::steady_clock, time.perf_counter (★★, =)

C++ **H01_11_sure_olcumu.cpp · satır 60–77**

```
60      // 1) Monotonik saat: std::chrono::steady_clock (geri gitmez)
61      auto t0 = std::chrono::steady_clock::now();
62      long long s1 = toplam(a);
63      auto t1 = std::chrono::steady_clock::now();
64      double duvar = std::chrono::duration<double>(t1 - t0).count();
65
66      // 2) İşlemci zamanı: std::clock() — süreç yalnızca CPU'da çalışırken
67      std::clock_t c0 = std::clock();
68      long long s2 = toplam(a);
69      std::clock_t c1 = std::clock();
70      double islemci = static_cast<double>(c1 - c0) / CLOCKS_PER_SEC;
71
72      // 3) zamanlayici modülü: 1 ısınma + 5 tekrar, ortanca. Sonuç
73      // dışarıdaki s3'e yazılır: kullanılmayan hesabı -O2 silebilir.
74      long long s3 = 0;
75      double orta = olc_ortanca([&] { s3 = toplam(a); }, 5);
76
77      std::cout << "n = " << n << ", tohum = 42, değerler 0..99\n";
```

Python **H01_11_sure_olcumu.py · satır 48–64**

```python
    # 2) İşlemci zamanı: time.process_time — süreç CPU'da çalışırken
    c0 = time.process_time()
    s2 = toplam(a)
    c1 = time.process_time()
    islemci = c1 - c0

    # 3) zamanlayici modülü: 1 ısınma + 5 tekrar, ortanca. Sonuç
    # dışarıdaki s3'e yazılır (nonlocal; C ve C++ ile aynı düzen).
    s3 = 0

    def gorev():
        nonlocal s3
        s3 = toplam(a)

    orta = olc_ortanca(gorev, 5)

    print(f"n = {n}, tohum = 42, değerler 0..99")
```

**DİL PENCERESİ · Farklılaşan satırlar**

Duvar saati: C'de `timespec_get(&t, TIME_UTC)` (C11), C++'ta monotonik `std::chrono::steady_clock`, Python'da `time.perf_counter()`; işlemci zamanı: `clock()` / `std::clock()` / `time.process_time()`. Ölçülen görev C'de `void *` veri alan bir fonksiyon, C++'ta lambda (`[&]`), Python'da iç fonksiyondur (`nonlocal`). stdout aynıdır; süreler stderr'dedir ve dile göre büyüklük mertebesi farklıdır.

| Dil | Satır | Kesit dışındaki farklı satır |
|---|---|---|
| Python | 43 | `t0 = time.perf_counter()` |

## A.12 Örnek 12 · Ölçüm tuzağı: sonucu kullanılmayan döngünün -O2'de kaybolması (★★★, =)

C++ **H01_12_olcum_tuzagi.cpp · satır 30–33**

```cpp
        s += a[i];
    }
    static_cast<void>(s);   // yalnızca derleyici uyarısını susturur
}
```

Python **H01_12_olcum_tuzagi.py · satır 29–35**

```python
def kullanilan_dongu(a):
    """Aynı döngü; sonuç modül düzeyindeki değişkene yazılır."""
    global sonuc_yeri
    s = 0
    for i in range(len(a)):
        s += a[i]
    sonuc_yeri = s          # ÇÖZÜM: sonucu kullan
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Yol haritasında Δ ipucu var, ama stdout yalnızca deterministik değerleri içerdiği için üç dilde aynıdır (=); fark süredir (stderr). gcc/g++ `-O2` sonucu kullanılmayan döngüyü tamamen siler (bu makinede ≈ 0 ms, eleman başına < 0,05 ns), `-O0`'da iki döngü aynı sürer; CPython ölü kodu silmez, iki döngü de ≈ 40 ns/eleman sürer.

| Dil | Satır | Kesit dışındaki farklı satır |
|---|---|---|
| C++ | 73 | `double t_bos = olc_ortanca([&] { kullanilmayan_dongu(a); }, 5);` |
| Python | 55 | `t_bos = olc_ortanca(lambda: kullanilmayan_dongu(a), 5)` |

## A.13 Örnek 13 · Deney raporu: n'e karşı süre tablosu (CSV) ve grafik (★★★, =)

C++ **H01_13_deney_raporu.cpp · satır 96–102**

```cpp
            rastgele_dizi(static_cast<std::size_t>(n), 0, 99);
        long long sonuc = 0;     // sonucu sakla: -O2 döngüyü silemesin
        double t = olc_ortanca([&] { sonuc = toplam(a); }, TEKRAR);
        std::cout << std::setw(10) << n << " " << std::setw(14) << sonuc
                  << " " << std::fixed << std::setprecision(4)
                  << std::setw(10)
                  << static_cast<double>(sonuc) / static_cast<double>(n)
```

Python **H01_13_deney_raporu.py · satır 55–75**

```python
    if csv_yolu is not None:
        try:
            csv = open(csv_yolu, "w", encoding="utf-8")
        except OSError:
            print(f"Hata: {csv_yolu} açılamadı", file=sys.stderr)
            return 1
        csv.write("n;dil;derleme;tekrar;ortanca_ms;toplam\n")

    print("Deney: dizi toplamı (Θ(n)), tohum 42, değerler 0..99")
    print(f"Düzen: 1 ısınma + {TEKRAR} tekrar, ortanca (süreler stderr/CSV)")
    print(f"{'n':>10} {'toplam':>14} {'ortalama':>10}")
    onceki = 0.0
    n = 1000
    while n <= nmax:
        rastgele_tohum(42)
        a = rastgele_dizi(n, 0, 99)
        sonuc = 0

        def gorev():
            nonlocal sonuc
            sonuc = toplam(a)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> CSV C'de `fopen`/`fprintf`, C++'ta `std::ofstream` ve `<<`, Python'da `open(..., encoding="utf-8")` ile yazılır; açılamazsa üç dilde de çıkış kodu 1. C/C++'ta `derleme` sütunu `__OPTIMIZE__` makrosundan (O0 / O1+) gelir; Python'da derleme düzeyi yoktur.

| Dil | Satır | Kesit dışındaki farklı satır |
|---|---|---|
| C++ | 77 | `std::ofstream csv;` |
| Python | 16 | `DERLEME = "CPython"   # Python'da derleme düzeyi yok; yorumlayıcının adı` |